Nios V ↔ Caliptra 2.x Integration Guide (passive / core-only)

> **Where the files are.** All paths in this guide are repo-relative (pwilliam282/caliptra-a5e-quartus):
>
> - 2.x glue RTL, sim, firmware, fuse script and Quartus project: `caliptra2x/` (`caliptra2x/rtl_2x/`, `caliptra2x/sim_2x/`, `caliptra2x/sim/`, `caliptra2x/fw_2x/`, `caliptra2x/scripts/`, `caliptra2x/quartus_a5e_2x/` core-only fit gate, `caliptra2x/quartus_a5e_2x_nios/` full Nios V + Caliptra 2.x system)
> - Upstream RTL: `external/caliptra-rtl-2x/` = chipsalliance/caliptra-rtl v2.1.2 (`49370266`) with `submodules/adams-bridge` (`b77e3d8`), a pinned git submodule, unmodified. Fetch it with `scripts/get_caliptra_a5e.sh` or `git clone --recursive` + `scripts/fetch_deps.sh`. See NIOS_CALIPTRA_2X_INTEGRATION.md §16.
> - Companion docs `P1_FIT_RESULT_2X_A5E.md` and `CALIPTRA_2X_INTEGRATION_PLAN.md`: this folder (`docs/caliptra-2x/`). The 1.x guide is `docs/NIOS_CALIPTRA_INTEGRATION.md`.
>
> caliptra-sw is not vendored (only the built ROM/bundle in `caliptra2x/fw_2x/`).

- **Project:** Caliptra 2.x in passive (core-only) mode on Intel Agilex 5 E **A5EC065BB32AE6S**, with a Nios V/m mailbox host (Avalon → AXI4).
- **RTL:** chipsalliance/caliptra-rtl **v2.1.2**, commit `49370266d12cb0c4a8f71b3a0ff7e54ba7d4866e`. `CALIPTRA_MODE_SUBSYSTEM` is **not** defined (core-only), and `CALIPTRA_INTERNAL_TRNG` is **not** defined (external TRNG).
- **Firmware:** caliptra-sw `dd1e79d5bf55a9bd5bdb05915757eaf34844145d`, which carries tags `rom-2.1.3-1` and `fw-2.1.4`. See FIRMWARE_AND_KEYS_2X.md.
- **Simulation:** Verilator 5.054, one cold boot to `ready_for_runtime`, **PASS**. See SIMULATION_2X.md.
- **Attestation / DPE in simulation:** ROM boot → FIRMWARE_LOAD → runtime, then 34 mailbox commands (IDevID/LDevID/FMC Alias/RT Alias certs, `STASH_MEASUREMENT`, PCR quotes, DPE GetProfile/CertifyKey/GetCertificateChain), **PASS** at 59,280,395 cycles with certificates byte-identical to the sw-emulator. Quick start: SIMULATION_2X.md §8.2. What the Nios model covers: §8.1.
- **Companion docs:** docs/caliptra-2x/P1_FIT_RESULT_2X_A5E.md (Quartus fit gate), docs/caliptra-2x/CALIPTRA_2X_INTEGRATION_PLAN.md (1.x → 2.x delta and phased plan). The 1.x guide this one is modelled on is `docs/NIOS_CALIPTRA_INTEGRATION.md` (rev 3).
- **Getting the sources / building:** §16 (clone with submodules; upstream RTL in `external/caliptra-rtl-2x/`) and §17 (Quartus Pro 25.3 build, Docker, license, measured results, known issues).
- **Nios V system:** the 2.x Platform Designer system `nios_caliptra2x_sys` is the 1.1 system with only the Caliptra window changed; side-by-side comparison in §18.
- **Date:** 2026-11-11 (America/Chicago).

> **Status.** This is a *design study*. The 2.x glue RTL and the boot flow are verified in Verilator. The design **does not fit** the A5E 065: synthesis estimates 279,685 / 222,400 ALMs (126 %) (§14). Every key, seed and entropy value is a **TEST/STUDY value**: the obfuscation key, CSR HMAC key, UDS, field entropy, the test-signed firmware, and the simulated TRNG data. The firmware is signed with caliptra-sw's **public fake keys**. See FIRMWARE_AND_KEYS_2X.md §6.

What exists and what doesn't (2.x tree):

Item	State
`caliptra2x/rtl_2x/nios_caliptra2x_top.sv`, `avmm_to_axi.sv`, `soc_fuse_ctrl2x.sv`, `soc_fw_store.sv`, `reset_sync.sv`, `caliptra_passive_top.sv`	Written; simulated (Verilator)

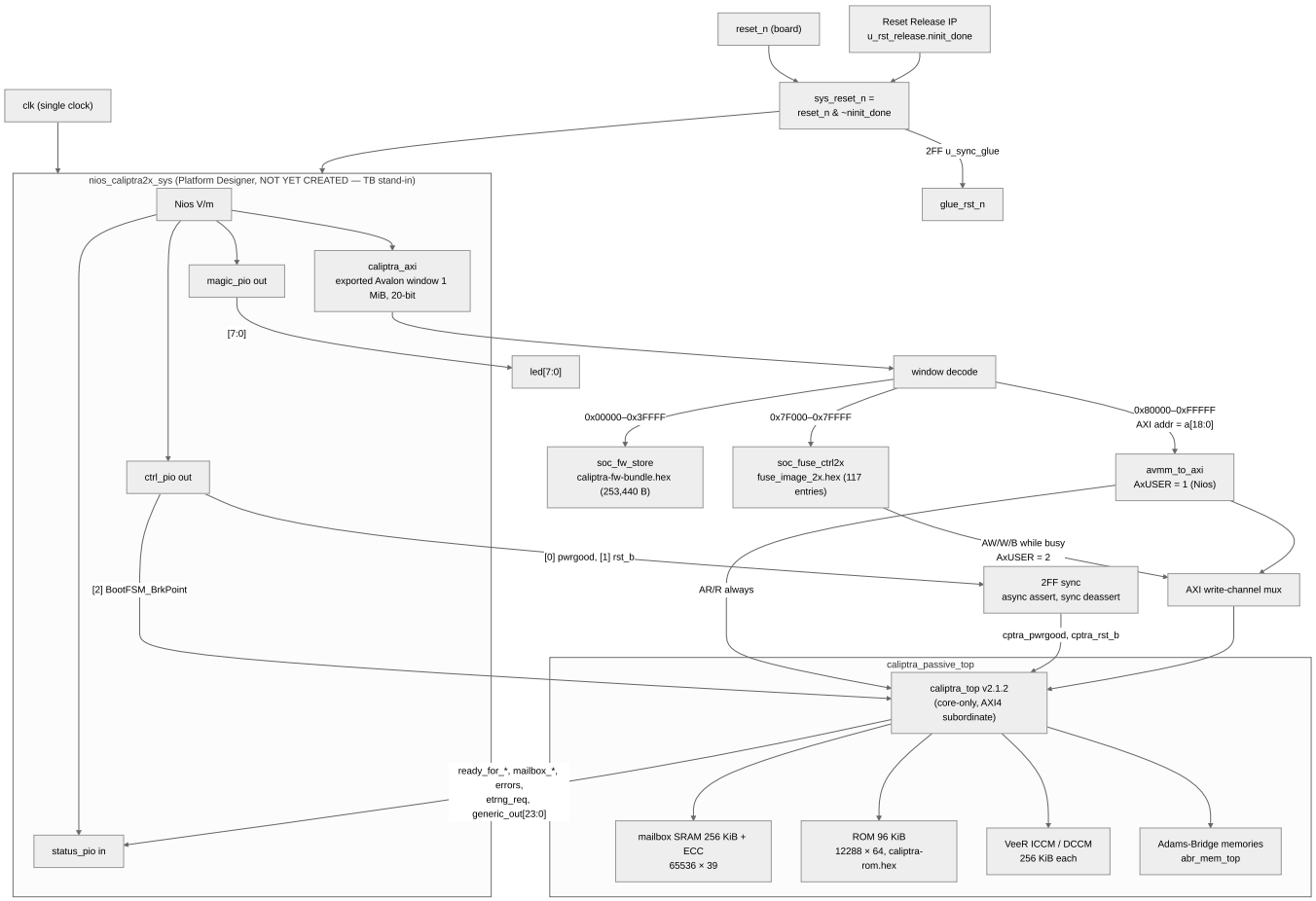
Item	State
`caliptra2x/scripts/gen_fuse_image_2x.py` → `caliptra2x/fw_2x/fuse_image_2x.hex`	Written; output used in the PASS run (regeneration is byte-identical)
`caliptra2x/fw_2x/` ROM + FW bundle (+ `build_rom.sh`, `SHA256SUMS`)	Built from caliptra-sw `dd1e79d`
Platform Designer system `nios_caliptra2x_sys` (Nios V + PIOs + 1 MiB window)	**Created** in `caliptra2x/quartus_a5e_2x_nios/qsys/` (`nios_caliptra2x_sys.tcl`, generated `.qsys` / `.ip` / output committed). Same Nios V/m, RAM, JTAG UART, PIOs, clock/reset and addresses as the 1.1 system (§18). Its exported ports match `nios_caliptra2x_top.sv` and the TB stand-in `nios_caliptra2x_sys` in `sim_2x/tb_nios2x_top_fw.sv` exactly: `clk_clk`, `reset_reset_n`, `caliptra_axi_address[19:0]`, `_read`, `_write`, `_writedata[31:0]`, `_byteenable[3:0]`, `_readdata[31:0]`, `_readdatavalid`, `_waitrequest`, `_burstcount`, `_debugaccess`, `ctrl_pio_export[31:0]` (out), `status_pio_export[31:0]` (in), `magic_pio_export[31:0]` (out)
Nios C firmware for 2.x (`main.c` port)	**Ported**: `caliptra2x/quartus_a5e_2x_nios/sw/app/main.c` (the §11 sequence; same LED/MAGIC codes, TRNG service, error handling, fatal recovery and warm-reset helper as 1.x). Builds (26,504 B text) into `onchip_mem.hex`. Not run on a Nios RTL model or a board; the Verilator TB still uses the behavioral host
Quartus project for the full Nios + 2.x top	**Created**: `caliptra2x/quartus_a5e_2x_nios/` (`nios_caliptra2x_a5e`, top `nios_caliptra2x_top`). Synthesis: §17. Fit overflows (does not fit the A5E 065)

1. Source files

File	Role
`caliptra2x/rtl_2x/nios_caliptra2x_top.sv`	FPGA top: Reset Release gate, PD system, SoC window decode, Avalon→AXI bridge, fuse controller, AXI write-channel mux, 2FF reset syncs, straps, status PIO
`caliptra2x/rtl_2x/caliptra_passive_top.sv`	Core-only wrapper around `caliptra_top`: flattens the AXI subordinate to ports, ties off the AXI manager (DMA) and all `ss_*` straps, instantiates the mailbox SRAM, ROM, VeeR ICCM/DCCM and Adams-Bridge memories
`caliptra2x/rtl_2x/avmm_to_axi.sv`	Avalon-MM agent → AXI4 manager, single beat, 1 outstanding, AxUSER = `user_i`
`caliptra2x/rtl_2x/soc_fuse_ctrl2x.sv`	SoC fuse sequencer (stand-in for OTP + fuse controller); replays `caliptra2x/fw_2x/fuse_image_2x.hex` over AXI with AxUSER 2
`caliptra2x/rtl_2x/soc_fw_store.sv`	Read-only M20K store holding `caliptra2x/fw_2x/caliptra-fw-bundle.hex` (stand-in for boot flash)
`caliptra2x/rtl_2x/reset_sync.sv`	`rst_sync_async_assert` / `pwrgood_sync_async_assert` (async assert, 2FF sync deassert)
`caliptra2x/rtl_2x/caliptra_veer_sram_export.sv`	VeeR ICCM/DCCM SRAM export
`caliptra2x/sim/rst_release_stub.sv`	Verilator stand-in for the Agilex 5 Reset Release IP (`ninit_done` high for 25 ns)
`caliptra2x/scripts/gen_fuse_image_2x.py`	Builds the fuse image from the FW bundle
`caliptra2x/fw_2x/`	ROM, FW bundle, hex files, `SHA256SUMS`, `build_rom.sh`, `fuse_image_2x.hex`
`caliptra2x/sim_2x/`	`boot2x.f`, `tb_nios2x_top_fw.sv`, `run_nios2x_top_fw.sh`, `BOOT_NIOS2X.md`, `nios2x_top_fw_run.log`
`caliptra2x/quartus_a5e_2x_nios/qsys/nios_caliptra2x_sys.tcl`	Platform Designer system: Nios V/m + 64 KiB RAM + JTAG UART + ctrl/status/magic PIOs + 1 MiB `caliptra_axi` mm_bridge (generated `.qsys`, `qsys/ip/`, `qsys/nios_caliptra2x_sys/` committed)

File	Role
`caliptra2x/quartus_a5e_2x_nios/sw/`	Nios host firmware `app/main.c`, HAL BSP `bsp/` (niosv-bsp), `build_app.sh`; image `../onchip_mem.hex`
`caliptra2x/quartus_a5e_2x_nios/nios_caliptra2x_a5e.qsf` (+ `gen_qsf.py`, `.sdc`, `ip/rst_release.ip`, `run_nios2x_a5e_docker.sh`)	Full-system Quartus project

2. Block diagram



3. Port and signal tables

3.1 `nios_caliptra2x_top` parameters and ports

Parameter	Default	Meaning
`ROM_INIT_FILE`	`"../fw_2x/caliptra-rom.hex"`	Caliptra ROM image (`$readmemh`, 64-bit words)
`FW_INIT_FILE`	`"../fw_2x/caliptra-fw-bundle.hex"`	FW bundle for `soc_fw_store` (32-bit LE words)
`FUSE_IMAGE`	`"../fw_2x/fuse_image_2x.hex"`	Fuse image for `soc_fuse_ctrl2x`
`FW_BYTES`	`253440`	Bundle size reported by fuse ctrl MMIO `0x10` (used as `MBOX_DLEN`)
`NUM_FUSES`	`117`	Image entries (104 fuse words + 12 owner-hash words + 1 lock)
`NIOS_AXI_USER`	`32'h1`	AxUSER driven on every Nios access
`FUSE_AXI_USER`	`32'h2`	AxUSER driven by the fuse controller

Port	Dir	Width	Connection
clk	in	1	Single clock (rising edge) for the PD system, glue and Caliptra. It is the only clock domain in the top, and both 2FF reset synchronizers run on it
reset_n	in	1	Board reset, **active-low**, asynchronous: it is ANDed with ~ninit_done (sys_reset_n = reset_n & ~ninit_done) without being synchronized first
led	out	8	magic_out[7:0] (magic_pio_export), active-high, 1 = bit set; driven by the magic PIO (TB model initializes it to 0)

3.2 Internal glue signals

Signal	Source → sink	Notes
ninit_done	rst_release → gate	Agilex 5 Reset Release IP output, **active-high**: 1 = device still initializing, 0 = user logic may run. Sim stub caliptra2x/sim/rst_release_stub.sv: 1 at time 0, 0 after 25 ns
sys_reset_n	reset_n & ~ninit_done	**Active-low**, combinational and asynchronous (not synchronized in this top). It drives the PD reset_reset_n, feeds u_sync_glue, and is ANDed into raw_pwrgood / raw_rst_b
glue_rst_n	u_sync_glue = rst_sync_async_assert #(.STAGES(2)) on sys_reset_n	**Active-low**. Assertion (1→0) is asynchronous; deassertion (0→1) comes 2 clk edges after sys_reset_n rises. It resets the read-valid flops of the window decode, avmm_to_axi, soc_fuse_ctrl2x and nios_axi_user
ctrl_out[0]	ctrl PIO → raw_pwrgood = ctrl_out[0] & sys_reset_n → u_sync_pwrgood = pwrgood_sync_async_assert #(.STAGES(2)) → cptra_pwrgood	**Active-high** (1 = power good). The cell sets cptra_pwrgood = 1 asynchronously as soon as raw_pwrgood rises, and drops it 2 clk edges after raw_pwrgood falls (see §10.1 note). The PIO reset value must be 0 (the TB model initializes ctrl = 0), so Caliptra starts unpowered
ctrl_out[1]	ctrl PIO → raw_rst_b = ctrl_out[1] & sys_reset_n → u_sync_rst_b = rst_sync_async_assert #(.STAGES(2)) → cptra_rst_b	**Active-low** reset (0 = Caliptra in reset). Assertion (1→0) is asynchronous; deassertion (0→1) comes 2 clk edges after raw_rst_b rises. Reset value 0. A warm reset drops only this bit
ctrl_out[2]	ctrl PIO → BootFSM_BrkPoint (direct, no synchronizer)	**Active-high**, reset value 0. soc_ifc_top.sv latches it once after cptra_noncore_rst_b deasserts. It only takes effect if debug is unlocked or the lifecycle is MANUFACTURING (or ss_debug_intent = 1). With the straps here (PRODUCTION, debug_locked = 1, ss_debug_intent = 0) it is **ignored**
status_in[31:0]	Caliptra outputs → status_pio_export (PIO input)	Bit map, polarity and reset values in §3.3
fuse_busy	soc_fuse_ctrl2x.busy	Active-high, combinational from the FSM state (0 in reset/IDLE). It is 1 in NEXT/SEND/RESP, when the fuse ctrl owns AW/W/B
bridge_idle	avmm_to_axi.idle	Active-high, 1 in bridge state IDLE (1 in reset). Meant for a warm-reset quiesce; **not yet used** in the top
nios_axi_user[31:0]	Flop on clk / glue_rst_n, reset and load value NIOS_AXI_USER = 32'h1	A register (not an assign) so the TB can force a wrong ID
generic_input_wires[63:0]	Constant 64'b0 → caliptra_top input	IS Table 13: bit 0 = zeroization status; with no source it is tied to 0

3.3 status_pio bits (32-bit input PIO, status_in)

nios_caliptra2x_top.sv: status_in = {generic_output_wires[23:0], etrng_req, cptra_error_non_fatal, cptra_error_fatal, mailbox_flow_done, mailbox_data_avail, ready_for_runtime, ready_for_mb_processing, ready_for_fuses}.

Common to every bit:

- **Direction:** `caliptra_top` **output** → `caliptra_passive_top` output → `status_in` → `status_pio_export` (PIO **input**, Nios reads it).
- **Polarity:** all **active-high**: 1 = condition true.
- **Clock domain:** Caliptra's `clk`. Each bit comes from a flop clocked by `clk` or by one of the clock-gated copies of `clk` that `clk_gate.sv` makes (`rdc_clk_cg`, `soc_ifc_clk_cg`):
  - bit 0: the boot FSM on `clk`;
  - bits 1, 2, 4, 7 and 31:8: `soc_ifc_reg` on `rdc_clk_cg`;
  - bits 5 and 6: `soc_ifc_top` flops on `rdc_clk_cg`;
  - bit 3: decoded combinationally from the mailbox FSM state register, which runs on `soc_ifc_clk_cg`. Nios, glue and Caliptra all run on the single top-level `clk`, so there is **no synchronizer**: the bits are wired straight to the PIO.
- **Reset value:** all **0**. Bits 0, 3, 5 and 6 reset on `cptra_noncore_rst_b`, the internal reset derived from `cptra_rst_b`. The register-backed bits (1, 2, 4, 7 and 31:8) have `resetsignal = cptra_rst_b`. So they all read 0 while `ctrl` = 0 after power-up.
- **Bit positions:** identical to the 1.x `quartus_a5e_nios/rtl/nios_caliptra_top.sv` status PIO. Only bit 1 changed name.

Bit	Signal (1-bit unless noted)	Driver inside caliptra-rtl v2.1.2	Reset	Meaning when 1 / host use	1.x bit
0	`ready_for_fuses`	`soc_ifc_boot_fsm.sv` flop on `clk`: `(boot_fsm_ps == BOOT_FUSE) && !fuse_wr_done_observed`	0	The boot FSM is in `BOOT_FUSE` and `CPTRA_FUSE_WR_DONE` has not been written yet, so fuses may be written. It is also wired to `soc_fuse_ctrl2x.ready_for_fuses`: START/START_DONE without it gives ERR, and it reads back as fuse ctrl STATUS bit 3. The host polls it at §11 step 3	bit 0 `ready_for_fuses`
1	`ready_for_mb_processing`	`CPTRA_FLOW_STATUS.ready_for_mb_processing` (bit 28), written by ROM/FW (`soc_ifc_reg`, `rdc_clk_cg`)	0	The ROM is ready for a mailbox command (FW_LOAD). The host polls it at step 7	bit 1 `ready_for_fw_push`
2	`ready_for_runtime`	`CPTRA_FLOW_STATUS.ready_for_runtime` (bit 29), written by FW	0	Runtime firmware is up. The host polls it at step 10	bit 2 `ready_for_runtime`
3	`mailbox_data_avail`	`mbox.sv` (`soc_ifc_clk_cg`) `soc_mbox_data_avail` = 1 while the mailbox FSM is in `MBOX_EXECUTE_SOC`	0 (FSM resets to `MBOX_IDLE`)	The Caliptra uC has set EXECUTE with data or a response for the SoC. The TB host does not poll it; it uses `MBOX_STATUS`	bit 3 `mailbox_data_avail`
4	`mailbox_flow_done`	`CPTRA_FLOW_STATUS.mailbox_flow_done` (bit 31), written by FW	0	FW reports that the mailbox flow is complete. The TB does not poll it. The final `CPTRA_FLOW_STATUS = b8000000` has it set	bit 4 `mailbox_flow_done`
5	`cptra_error_fatal`	`soc_ifc_top.sv` flop on `rdc_clk_cg`, set by a new unmasked bit in `CPTRA_HW_ERROR_FATAL` / `CPTRA_FW_ERROR_FATAL`	0	Fatal error. It is **sticky**: the RTL has no clear path except reset (the RTL comment says the platform must apply `cptra_rst_b`). The IS recovery rule is a pwrgood cycle (§10.3). The TB treats it as FAIL	bit 5 `cptra_error_fatal`
6	`cptra_error_non_fatal`	`soc_ifc_top.sv` flop on `rdc_clk_cg`, set like bit 5 from the `*_NON_FATAL` registers	0	Non-fatal error. It deasserts once every unmasked non-fatal bit is cleared; the host clears them with RW1C	bit 6 `cptra_error_non_fatal`
7	`etrng_req`	`soc_ifc_top.sv` `trng_req = CPTRA_TRNG_STATUS.DATA_REQ` (no `CALIPTRA_INTERNAL_TRNG`)	0	ROM/FW requests TRNG data. The host writes `CPTRA_TRNG_DATA[0..11]`, then `DATA_WR_DONE` (§9)	bit 7 `etrng_req`

Bit	Signal (1-bit unless noted)	Driver inside caliptra-rtl v2.1.2	Reset	Meaning when 1 / host use	1.x bit
31:8	`generic_output_wires[23:0]` (24 of 64 bits)	`CPTRA_GENERIC_OUTPUT_WIRES[0][23:0]`, written by ROM/FW (`[63:24]` not wired to the PIO)	0	ROM/FMC/RT console: `[7:0]` = ASCII character, and `[8]` toggles once per character (the TB decoder in `tb_nios2x_top_fw.sv` prints lines from these). `[23:9]` are not decoded	bits 31:8 `generic_output_wires[23:0]`

3.4 `caliptra_passive_top` / `caliptra_top` connections

Widths come from `config_defines.svh`: `CALIPTRA_AXI_DATA_WIDTH` = 32, `CALIPTRA_AXI_USER_WIDTH` = 32, `CALIPTRA_AXI_ID_WIDTH` = 8. The SoC-interface address width is `CALIPTRA_SLAVE_ADDR_WIDTH(SOC_IFC)` = **19** bits (base `0x3000_0000`, mask `0x3007_FFFF`).

Clocks, resets, straps, keys

`caliptra_top` port	Width	Driven by (`nios_caliptra2x_top` → wrapper)	Status
`clk`	1	Top-level `clk`, direct	Caliptra input; same clock as Nios and glue, so there is no CDC
`cptra_pwrgood`	1	`u_sync_pwrgood` output (from `ctrl_out[0] & sys_reset_n`)	Input, **active-high**; 0 = Caliptra powered off (cold reset of all pwrgood-domain state). Reset value 0
`cptra_rst_b`	1	`u_sync_rst_b` output (from `ctrl_out[1] & sys_reset_n`)	Input, **active-low**; 0 = Caliptra in (warm) reset. Async assert, 2FF sync deassert. Reset value 0
`cptra_obf_key`	256	`STUDY_PROD_MIMIC_OBF_KEY` = `31358e8a_f34d6a45_…_b2c3b8d5` (localparam)	**TEST-ONLY**
`cptra_csr_hmac_key`	16 × 32 (`CLP_CSR_HMAC_KEY_DWORDS` = 16)	`{16{32'h5A3C_96E1}}`	**TEST-ONLY**
`security_state.device_lifecycle`	2	`2'b11` = DEVICE_PRODUCTION	**TEST-ONLY** (hardwired)
`security_state.debug_locked`	1	`1'b1`	**TEST-ONLY** (hardwired)
`cptra_obf_field_entropy_vld` / `cptra_obf_field_entropy`	1 / 256	`0` / `'0`	Passive: tie 0 (IS Table 12)
`cptra_obf_uds_seed_vld` / `cptra_obf_uds_seed`	1 / 512	`0` / `'0`	Passive: tie 0
`BootFSM_BrkPoint`	1	`ctrl_out[2]`, direct	Input, active-high, reset 0. Ignored with the PRODUCTION + debug-locked straps (§3.2)
`scan_mode`	1	`1'b0`	
`jtag_tck/tms/tdi/trst_n`	1 each	`0` (TAP held in reset)	OPEN (no TAP endpoint)
`jtag_tdo`, `jtag_tdoEn`	1 each	unconnected	

AXI4 subordinate (`s_axi_w_if`, `s_axi_r_if`), flattened in the wrapper

Channel	Signals	Driven by
AW	`awaddr[18:0]`, `awburst`, `awsize`, `awlen`, `awuser[31:0]`, `awid[7:0]`, `awlock`, `awvalid` / `awready`	Mux: fuse ctrl while `fuse_busy`, else bridge. `awburst/awsize/awlen/awid/awlock` always come from the bridge (constants INCR / 2 / 0 / 0 / 0)

Channel	Signals	Driven by
W	`wdata[31:0]`, `wstrb`, `wuser`, `wvalid` / `wready`, `wlast`	Mux for `wdata/wvalid/wuser`; `wstrb=4'hF`, `wlast=1` from bridge
B	`bresp`, `bid`, `buser`, `bvalid` / `bready`	`bresp/bvalid` to both; `bready` muxed; `bid/buser` unconnected
AR	`araddr`, `arburst`, `arsize`, `arlen`, `aruser`, `arid`, `arlock`, `arvalid` / `arready`	Bridge only
R	`rdata`, `rresp`, `rid`, `ruser`, `rlast`, `rvalid` / `rready`	Bridge only; `rid/ruser/rlast` unconnected

AXI manager (DMA), subsystem straps and other ports, all tied off in the wrapper

Port group	Tie-off	Reason
`m_axi_w_if`, `m_axi_r_if`	`awready/wready/arready/bvalid/rvalid = 0`, data/resp/id/user = 0	Passive: DMA AXI manager must not reach the SoC
`strap_ss_caliptra_base_addr`, `strap_ss_mci_base_addr`, `strap_ss_recovery_ifc_base_addr`, `strap_ss_external_staging_area_base_addr`, `strap_ss_otp_fc_base_addr`, `strap_ss_uds_seed_base_addr`, `strap_ss_key_release_base_addr`, `strap_ss_key_release_key_size`, `strap_ss_prod_debug_unlock_auth_pk_hash_reg_bank_offset`, `strap_ss_num_of_prod_debug_unlock_auth_pk_hashes`, `strap_ss_caliptra_dma_axi_user`, `strap_ss_strap_generic_0..3`	`'0`	IS Table 11: subsystem only; tie 0 in passive
`ss_debug_intent`	`1'b0`	Subsystem only
`ss_ocp_lock_en`	`1'b0` (constant)	OCP LOCK needs subsystem; the runtime log confirms `[rt] OCP LOCK Unsupported`
`recovery_data_avail`, `recovery_image_activated`	`1'b0`	Subsystem recovery interface
`ss_dbg_manuf_enable`, `ss_soc_dbg_unlock_level`, `ss_generic_fw_exec_ctrl`	unconnected outputs	
`trace_rv_*`	unconnected	
`itrng_data[3:0]`, `itrng_valid`	`4'h0`, `1'b0`	Internal TRNG not compiled in
`generic_input_wires[63:0]`	`64'b0`	
`generic_output_wires[63:0]`	`[23:0] → status_pio[31:8]`	Log channel

Status and interrupts

Port	Width	To
`ready_for_fuses`, `ready_for_mb_processing`, `ready_for_runtime`	1 each	`status_pio[0..2]`; `ready_for_fuses` also to the fuse ctrl
`mailbox_data_avail`, `mailbox_flow_done`	1 each	`status_pio[3..4]` (polled, no IRQ)
`cptra_error_fatal`, `cptra_error_non_fatal`	1 each	`status_pio[5..6]` (polled, no IRQ)
`etrng_req`	1	`status_pio[7]`

Memories (instantiated inside `caliptra_passive_top`)

Memory	Shape	Module
Mailbox SRAM	`CPTRA_MBOX_DEPTH` × `CPTRA_MBOX_DATA_AND_ECC_W` = 65536 × 39 (256 KiB data + 7-bit ECC; passive size)	`caliptra_sram`

Memory	Shape	Module
ROM (imem)	`CALIPTRA_IMEM_DEPTH` × 64 = 12288 × 64 (96 KiB, `CALIPTRA_IMEM_BYTE_SIZE` = 98304)	`caliptra_rom_m20k`, `$readmemh(ROM_INIT_FILE)`
VeeR ICCM / DCCM	256 KiB each	`caliptra_veer_sram_export`
Adams-Bridge (ML-DSA / ML-KEM)	10 memories via `abr_mem_if`	`abr_mem_top`

---

4. AXI user IDs (AxUSER)

Agent	AxUSER	Where set	Programmed into
Nios (bridge)	`0x0000_0001`	`NIOS_AXI_USER` → `nios_axi_user` → `avmm_to_axi.user_i` (AWUSER, WUSER, ARUSER)	`CPTRA_MBOX_VALID_AXI_USER[0]` + lock; `CPTRA_TRNG_VALID_AXI_USER` + lock
Fuse controller	`0x0000_0002`	`FUSE_AXI_USER` → `soc_fuse_ctrl2x.axi_user` (AWUSER, WUSER)	`CPTRA_FUSE_VALID_AXI_USER` + lock
(reserved)	`0xFFFF_FFFF`	never driven	IS Table 19: "CANNOT use value 0xFFFFFFFF". It is also the RTL default `CPTRA_DEF_MBOX_VALID_AXI_USER`

- The IDs are unique per agent and non-zero, as IS Table 19 recommends.
- The RTL integration parameters are left at upstream defaults: `CPTRA_SET_MBOX_AXI_USER_INTEG = 0`, `CPTRA_SET_FUSE_AXI_USER_INTEG = 0`. The IDs are therefore *programmed and locked* by the host on every cold boot (§11, step 4), not hardened at compile time.
- **Fuse range gating.** In `soc_ifc_arb.sv`, SoC requests to `0x30200–0x305FF` (`SOC_IFC_FUSE_START_ADDR`..`_END_ADDR`) are routed only if `valid_fuse_user` is set. Once `CPTRA_FUSE_AXI_USER_LOCK` = 1, that means AxUSER == `CPTRA_FUSE_VALID_AXI_USER` (= 2). So **the Nios (user 1) can neither write nor read fuse registers**. This explains the `FUSE_PQC_KEY_TYPE readback=00000000` seen in the sim (SIMULATION_2X.md §4.3).
- `CPTRA_OWNER_PK_HASH[12]` (`0x30140`) and `CPTRA_OWNER_PK_HASH_LOCK` (`0x30170`) are **outside** the fuse range. Any SoC requester can write them until the lock is set. In this design the fuse controller writes and locks them. Note that the Nios could also write them before the lock (see checklist, §13).
- The sim's `+test=axiuser` checks: a mailbox write with AxUSER 3 gives SLVERR; a fuse write with AxUSER 3 gives SLVERR; a fuse write with AxUSER 1 (Nios) gives SLVERR; a `HW_REV_ID` read with AxUSER 1 is accepted (`TB_AXIUSER PASS`).

---

5. Memory map

5.1 Nios SoC window (20-bit, 1 MiB) — decode in `nios_caliptra2x_top`

The window (`caliptra_win`, exported as `caliptra_axi`) sits at Nios address **`0x0010_0000–0x001F_FFFF`** in `nios_caliptra2x_sys` (`CALIPTRA_WIN_BASE` in the BSP `system.h`). Offsets below are **window offsets** (`cl_address[19:0]`); the Nios address is `0x0010_0000 + offset` (e.g. `mbox_csr` `0x001A_0000`, `soc_ifc` `0x001B_0000`, fuse ctrl `0x0017_F000`).

Window offset	Decode (RTL)	Target
`0x00000–0x3FFFF`	`!a[19] && !a[18]`	`soc_fw_store`: 65536 × 32-bit words (256 KiB), read-only, 1-cycle read. The 2.x bundle uses 63,360 words (253,440 B)
`0x40000–0x7EFFF`	—	Unmapped: no waitrequest; read data is not meaningful
`0x7F000–0x7FFFF`	`!a[19] && a[18:12]==7'h7F`	`soc_fuse_ctrl2x` MMIO (§7.2)
`0x80000–0xFFFFF`	`a[19]`	Caliptra AXI subordinate; **AXI address = `a[18:0]`**

The RTL comment in `nios_caliptra2x_top.sv` says "0x80000-0xBFFFF … AXI addr = cl_address[17:0]". The actual decode is `sel_cptra = cl_address[19]`, and the bridge takes `cl_address[AXI_AW-1:0]` with `AXI_AW` = 19. So the whole upper half maps through. Caliptra's targets all sit below AXI `0x40000` for the SoC (see 5.2), so the narrower comment is harmless, but this table follows the RTL.

5.2 Caliptra 2.x SoC AXI map (from `caliptra_reg_defines.svh` / `soc_ifc_pkg.sv`, v2.1.2)

Internal addresses are `0x3000_0000 + AXI address` (`SOC_IFC_REG_OFFSET`).

AXI address	Nios window offset	Block	SoC access
`0x20000–0x20FFF`	`0xA0000`	`mbox_csr`	Valid mailbox AxUSERs
`0x21000–0x21FFF`	`0xA1000`	SHA512 accelerator CSR	Only the `SS_CALIPTRA_DMA_AXI_USER` → not usable by the Nios in passive mode
`0x22000–0x22FFF`	`0xA2000`	AXI DMA registers	Present in RTL; DMA unused in passive mode
`0x30000–0x3FFFF`	`0xB0000`	`soc_ifc` registers	Any requester except the fuse range
`0x30200–0x305FF`	`0xB0200`	Fuse registers (subset of `soc_ifc`)	Only `CPTRA_FUSE_VALID_AXI_USER` once locked

5.3 Registers used by the host (verified against `caliptra_reg_defines.svh`)

Register	AXI	Window	Use
`MBOX_LOCK`	`0x20000`	`0xA0000`	Read to acquire (bit 0 = 0 → acquired)
`MBOX_USER`	`0x20004`	`0xA0004`	Expect `0x1`
`MBOX_CMD`	`0x20008`	`0xA0008`	`0x46574C44` = FIRMWARE_LOAD
`MBOX_DLEN`	`0x2000C`	`0xA000C`	253,440
`MBOX_DATAIN`	`0x20010`	`0xA0010`	Full-word writes only
`MBOX_DATAOUT`	`0x20014`	`0xA0014`	(not used for FW load)
`MBOX_EXECUTE`	`0x20018`	`0xA0018`	1 = execute, 0 = release
`MBOX_STATUS`	`0x2001C`	`0xA001C`	`[3:0]` 2 = CMD_COMPLETE
`CPTRA_HW_ERROR_FATAL` / `_NON_FATAL`	`0x30000` / `0x30004`	`0xB0000` / `0xB0004`	RW1C
`CPTRA_FW_ERROR_FATAL` / `_NON_FATAL`	`0x30008` / `0x3000C`	`0xB0008` / `0xB000C`	RW1C
`CPTRA_BOOT_STATUS`	`0x30038`	`0xB0038`	
`CPTRA_FLOW_STATUS`	`0x3003C`	`0xB003C`	
`CPTRA_SECURITY_STATE`	`0x30044`	`0xB0044`	Sim read `0x00000007` = Production + debug locked
`CPTRA_MBOX_VALID_AXI_USER[0]` / `CPTRA_MBOX_AXI_USER_LOCK[0]`	`0x30048` / `0x3005C`	`0xB0048` / `0xB005C`	1 / 1
`CPTRA_TRNG_VALID_AXI_USER` / `CPTRA_TRNG_AXI_USER_LOCK`	`0x30070` / `0x30074`	`0xB0070` / `0xB0074`	1 / 1
`CPTRA_TRNG_DATA[0..11]`	`0x30078–0x300A4`	`0xB0078–0xB00A4`	12 entropy words
`CPTRA_TRNG_CTRL`	`0x300A8`	`0xB00A8`	(not used)
`CPTRA_TRNG_STATUS`	`0x300AC`	`0xB00AC`	Write 2 = `DATA_WR_DONE`
`CPTRA_FUSE_WR_DONE`	`0x300B0`	`0xB00B0`	Written 1 by the fuse ctrl (its `FUSE_WR_DONE` parameter `32'h0003_00B0`)

Register	AXI	Window	Use
CPTRA_TIMER_CONFIG	0x300B4	0xB00B4	20000 (ps period for 50 MHz)
CPTRA_BOOTFSM_GO	0x300B8	0xB00B8	(only with BootFSM_BrkPoint=1)
CPTRA_HW_REV_ID	0x300D4	0xB00D4	Sim read 0x00000212
CPTRA_HW_CONFIG	0x300E0	0xB00E0	Sim read 0x00000010 (iTRNG_en=0, SUBSYSTEM_MODE_en=0)
CPTRA_FUSE_VALID_AXI_USER / CPTRA_FUSE_AXI_USER_LOCK	0x30108 / 0x3010C	0xB0108 / 0xB010C	2 / 1
CPTRA_WDT_CFG[0] / [1]	0x30110 / 0x30114	0xB0110 / 0xB0114	100,000,000 / 0
CPTRA_OWNER_PK_HASH[0..11]	0x30140–0x3016C	—	Written by the fuse ctrl
CPTRA_OWNER_PK_HASH_LOCK	0x30170	0xB0170	Written 1 by the fuse ctrl

---

6. avmm_to_axi bridge

- **Avalon side:** waitrequest + readdatavalid agent, with no bursts and no pipelining. avs_byteenable is ignored because Caliptra SoC registers are 32-bit.
- **AXI side:** single beat, AxLEN=0, AxSIZE=2 (32-bit), AxBURST=INCR (2'b01), AxLOCK=0, AxID=0, WSTRB=4'hF, WLAST=1. AWUSER = WUSER = ARUSER = user_i. The address is word-aligned ({a[AW-1:2],2'b00}).
- **One outstanding transaction.** States: IDLE → WADDR → WRESP → DONE for writes and IDLE → RADDR → RDATA → DONE → RESP for reads. AW and W are issued together, and each drops on its own handshake.
- **Avalon handshake:** avs_waitrequest is low for exactly one cycle (DONE) to accept. Read data comes one cycle later with avs_readdatavalid (RESP).
- **Errors:** BRESP/RRESP ≠ OKAY sets err_q, so avs_response = 2'b10 (SLVERR) for the last transaction. The TB reads u_bridge.err_q directly for the AxUSER test. **In nios_caliptra2x_top.sv the bridge's avs_response drives cl_response, but cl_response is not connected to any nios_caliptra2x_sys port.** So SLVERR does not reach the Nios today. Adding a caliptra_axi_response port to the PD window is an open item (§15).
- **Spec fit (IS "Unsupported features"):** single outstanding, in order, no narrow writes to mbox_datain / TRNG_DATA. AxCACHE/PROT/REGION/QOS are not driven.
- **idle output:** high in IDLE, intended for warm-reset quiescing. It is not yet used by the top.

---

7. soc_fuse_ctrl2x — fuse sequencer

7.1 Image format (caliptra2x/fw_2x/fuse_image_2x.hex)

- Each line is one $readmemh 64-bit word, {AXI byte address[31:0], data[31:0]}, for example the first line 00030200811944f5 (FUSE_UDS_SEED[0]) and the last line 0003017000000001 (CPTRA_OWNER_PK_HASH_LOCK = 1). Addresses are explicit because the 2.x fuse map has holes.
- The controller truncates the address to AW = 19 bits.
- 117 entries: 104 fuse words, then 12 CPTRA_OWNER_PK_HASH words, then CPTRA_OWNER_PK_HASH_LOCK = 1. The controller then appends CPTRA_FUSE_WR_DONE = 1 itself (addr = FUSE_WR_DONE = 0x300B0). That makes **118 AXI writes**.

7.2 MMIO (window 0x7F000, 4 KiB, word registers)

Offset	Name	Access	Bits
0x00	ID	RO	0x4655_5345 ("FUSE")
0x04	CTRL	WO	bit0 **START**, bit1 **CLEAR** (clears DONE/ERR), bit2 **START_DONE**

Offset	Name	Access	Bits
`0x08`	STATUS	RO	bit0 BUSY, bit1 DONE, bit2 ERR, bit3 READY_FOR_FUSES
`0x0C`	COUNT	RO	AXI writes completed (118 after a full START)
`0x10`	FW_BYTES	RO	`FW_BYTES` parameter (253,440)
`0x14`	NUM_FUSES	RO	`NUM_FUSES` parameter (117)

7.3 Behaviour

- **START** (cold boot) replays entries 0..116, then `FUSE_WR_DONE`.
- **START_DONE** (warm reset) starts with `idx = NUM_FUSES`, which is "last", so it writes **only** `FUSE_WR_DONE = 1`. Fuse registers and `CPTRA_OWNER_PK_HASH*` are pwrgood-domain and persist across a warm reset. START wins if both bits are set.
- START or START_DONE while `ready_for_fuses` = 0 leads to `ERR`, with no writes.
- Any write with `BRESP` ≠ OKAY leads to `ERR`, and the sequence stops (`COUNT` shows how far it got).
- While `busy` is set (NEXT/SEND/RESP), the top muxes AW/W/B to the fuse controller. The Nios only polls the fuse ctrl STATUS (a local slave) meanwhile, so the bridge is idle. The read channels always belong to the bridge.
- Measured in sim: START issued just after cycle 136, `STATUS=00000002 COUNT=118` at cycle 614, so about 4 cycles per write.

---

8. Fuse map decode (what `gen_fuse_image_2x.py` writes)

Offsets are verified against `caliptra_reg_defines.svh` (`CLP_SOC_IFC_REG_*`, minus `0x3000_0000`). The access column is from `soc_ifc_fuse_reg.rdl` / `soc_ifc_external_reg.rdl`.

AXI addr	Register	Words	Value written	Notes
`0x30200`	`FUSE_UDS_SEED[0..15]`	16	STUDY: `0x1F2E3D4C ^ (0x9E3779B9·(i+1))` → `811944f5 2340ce3e … fc59a6dc`	**512-bit** in 2.x (384 in 1.x). Caliptra: no access; SoC: WL-S (write-only). **TEST-ONLY**
`0x30240`	`FUSE_FIELD_ENTROPY[0..7]`	8	STUDY: `0xA7C3E519 ^ (0x85EBCA6B·(i+1))` → `22282f72 ac1471cf … 889db641`	256-bit. **TEST-ONLY**
`0x30260`	`FUSE_VENDOR_PK_HASH[0..11]`	12	SHA-384(`preamble.vendor_pub_key_info`) as 12 big-endian words → `30399676 a17e3e97 3677b3ff 862f4bf2 d1932d88 4778453c 376fe00d c93fb8aa 0770f3eb f3411a08 53e9c57e ce8a2980`	Hash of the ECC + PQC key *descriptors* (1,736 B). ROM rejects an all-zero value
`0x30290`	`FUSE_ECC_REVOCATION`	1	0	4-bit field
`0x302B4`	`FUSE_FMC_KEY_MANIFEST_SVN`	1	0	
`0x302B8`	`FUSE_RUNTIME_SVN[0..3]`	4	0	Bundle header SVN must be ≥ fuse SVN; the bundle SVN is 0
`0x302C8`	`FUSE_ANTI_ROLLBACK_DISABLE`	1	0	
`0x302CC`	`FUSE_IDEVID_CERT_ATTR[0..23]`	24	0	

AXI addr	Register	Words	Value written	Notes
0x3032C	FUSE_IDEVID_MANUF_HSM_ID[0..3]	4	0	
0x30340	FUSE_LMS_REVOCATION	1	0	
0x30344	FUSE_MLDSA_REVOCATION	1	0	
0x30348	FUSE_SOC_STEPPING_ID	1	1	Test value
0x3034C	FUSE_MANUF_DBG_UNLOCK_TOKEN[0..15]	16	0	Debug unlock is subsystem-only
0x3038C	FUSE_PQC_KEY_TYPE	1	**1** = manifest pqc_key_type	RDL: 2-bit, "one-hot", bit0 = MLDSA, bit1 = LMS. The ROM compares it with FwVerificationPqcKeyType::from_u8() (MLDSA = 1, LMS = 3). Skipped only when Unprovisioned
0x30390	FUSE_SOC_MANIFEST_SVN[0..3]	4	0	Unused in passive
0x303A0	FUSE_SOC_MANIFEST_MAX_SVN	1	0	
0x303C0	FUSE_HEK_SEED[0..7]	8	0	OCP LOCK; unused (ss_ocp_lock_en =0)
	Fuse words subtotal	**104**		Holes at 0x294–0x2B0 and 0x3A4–0x3BC are skipped
0x30140	CPTRA_OWNER_PK_HASH[0..11]	12	SHA-384(preamble.owner_pub_keys) → 48afdb07 3c5e0d4e e4649046 8ef81f2c f57249b6 e76a28f5 fca4de69 6a7d3e2e d3efc4e6 77431854 3e95307a 54988bd7	**Not a fuse in 2.x.** Architectural register, resetsignal = cptra_pwrgood, SoC RWL-S
0x30170	CPTRA_OWNER_PK_HASH_LOCK	1	1	RW1-S; reset only by power cycle
0x300B0	CPTRA_FUSE_WR_DONE	(1)	1	Appended by the controller, not in the image

Owner key and entropy, in plain words:

- The **owner key hash is provisioned**, written to CPTRA_OWNER_PK_HASH and locked. The sim read back [0]=48afdb07 and LOCK=1. The ROM then verified the image against it.
- **UDS and field entropy are provisioned** with STUDY values (not secrets). The SoC cannot read them back (write-only), and the Nios can't access the fuse range anyway.
- There is no FUSE_LIFE_CYCLE in 2.x. Lifecycle comes only from the security_state straps (Production, debug locked).

---

9. TRNG (external, serviced by the host)

9.1 Hardware path

- CALIPTRA_INTERNAL_TRNG is not defined. CPTRA_HW_CONFIG.iTRNG_en = 0. itrng_* are tied 0.
- When Caliptra firmware wants entropy, it raises etrng_req (status_pio[7]).

- The host writes 12 words to `CPTRA_TRNG_DATA[0..11]` (`0x30078–0x300A4`), then writes `CPTRA_TRNG_STATUS = 2` (`DATA_WR_DONE`, `0x300AC`).
- Writes must come from `CPTRA_TRNG_VALID_AXI_USER` (= 1, locked). `DATA_WR_DONE` is hardware-cleared when Caliptra FW clears `DATA_REQ` (`soc_ifc_top.sv`).

9.2 Entropy source — TEST-ONLY

- **Sim:** the TB host model writes `$urandom` values. In the 1.x Nios firmware, `main.c` uses `xorshift32` seeded `0x1234ABCD`. The 2.x Nios firmware port would keep that (it isn't ported yet).
- Neither source is a real entropy source. IS §TRNG: *"It is a requirement for FIPS compliance that this TRNG HW API is always handled by SoC hardware gasket logic (and not some SoC ROM or firmware code)."*
- **Production options:** a hardware ETRNG gasket fed by a physical noise source, or `CALIPTRA_INTERNAL_TRNG` with a PTRNG on `itrng_data/itrng_valid`.

9.3 Sim numbers

- **217** requests serviced in one boot.
- The first was at cycle 379,157.
- 21 had been serviced by `ready_for_mb_processing`.
- The TB fails if `etrng_req` stays high for 100,000 cycles after `DATA_WR_DONE`; that never happened.

---

10. Reset, power-good, watchdog, errors

10.1 Resets

Event	How	Effect
FPGA configuration	Reset Release IP holds `ninit_done` =1 → `sys_reset_n` =0	PD system in reset (`reset_reset_n` =0). `glue_rst_n` =0 (async). `raw_pwrgood` = `raw_rst_b` =0, so `cptra_rst_b` =0 (async) and `cptra_pwrgood` =0 (2 `clk` after; it was never raised)
Board reset	`reset_n` =0 → `sys_reset_n` =0	PD system reset; the ctrl PIO returns to 0. `glue_rst_n` =0 (async). `cptra_rst_b` =0 (async), and `cptra_pwrgood` falls 2 `clk` edges later (needs `clk` running). The result is a Caliptra **cold** reset
Caliptra cold reset	Host clears `ctrl[0]` (and `[1]`), then sets `ctrl=1` → wait → `ctrl=3`	pwrgood-domain state cleared (fuses, owner hash + lock, AxUSER locks) → full fuse push needed
Caliptra warm reset	Host clears only `ctrl[1]`, then sets it again	Fuses/owner hash persist; host must issue fuse ctrl **START_DONE** in `BOOT_FUSE`. **Not simulated in 2.x**

- Both Caliptra resets use `reset_sync.sv` cells (2 stages, `SYNCHRONIZER_IDENTIFICATION FORCED_IF_ASYNCHRONOUS`):
  - `cptra_rst_b` (active-low): 1→0 is asynchronous, and 0→1 is synchronized (2 `clk`).
  - `cptra_pwrgood` (active-high): 0→1 is asynchronous, and 1→0 is synchronized (2 `clk`). This follows IS Table 4 literally ("Asynchronous Assertion, Synchronous deassertion to clk").
- **Note on pwrgood.** The IS RDC table also lists `CPTRA_PWRGD` as an async, active-low reset. Seen that way, the 0→1 edge of `cptra_pwrgood` is the reset *release*, and this cell does not synchronize it. Here that edge always comes from the ctrl PIO flop, which is clocked by `clk` (`sys_reset_n` cannot raise it, because the PIO is 0 when `sys_reset_n` rises), so it is already synchronous. Keep that property if the source changes.
- In the sim, pwrgood is raised and then `rst_b` follows 20 cycles later (`cold_release()`). That is above IS Table 19's 10-clock minimum.

10.2 Watchdog

- The host writes `CPTRA_WDT_CFG[0] = 100,000,000` and `CPTRA_WDT_CFG[1] = 0` before fuses.

- The debug-locked ROM uses them: the log shows `[state] Starting the WD Timer 100000000 cycles` at cycles 384,577 and 450,858.
- There are no WDT pins. A cascaded timeout shows up as `cptra_error_fatal`.

10.3 Errors

- `cptra_error_fatal` / `_non_fatal` are polled on `status_pio[5:6]`.
- The TB host clears non-fatal errors by RW1C (`HW/FW_ERROR_NON_FATAL`), and treats any fatal as a test FAIL. It dumps BOOT/FLOW status and the error registers.
- **The spec rule is to recover from a fatal error only by a pwrgood cycle.** The 1.x firmware does this (≤3 retries). The 2.x TB does **not** exercise fatal recovery.
- At `ready_for_runtime`, every HW/FW fatal and non-fatal register was 0, and `CPTRA_FLOW_STATUS = b8000000`.

---

11. Boot sequence



Host sequence, step by step. This is the TB behavioral host `nios_caliptra2x_sys`. `caliptra2x/quartus_a5e_2x_nios/sw/app/main.c` implements the same sequence on the Nios (it waits on `status_pio` with polling loops instead of fixed clock counts).

1. Wait until the Reset Release IP releases `sys_reset_n`.
2. `ctrl = 1` (pwrgood), wait 20 clk, `ctrl = 3` (rst_b).
3. Poll `status[0]` (`ready_for_fuses`). Read `HW_REV_ID` (expect `0x212`), `HW_CONFIG` (expect `iTRNG_en=0`, `SUBSYSTEM_MODE_en=0`) and `CPTRA_SECURITY_STATE` (expect `7`).
4. Program and lock the AxUSERs: `FUSE_VALID_AXI_USER=2` + lock, `TRNG_VALID_AXI_USER=1` + lock, `MBOX_VALID_AXI_USER[0]=1` + lock. Then `TIMER_CONFIG=20000` and `WDT_CFG0=100,000,000`, `WDT_CFG1=0`.
5. Fuse ctrl: check ID `0x46555345`, write `CTRL=1` (START), poll `STATUS` until !BUSY and (DONE|ERR). Expect `COUNT=118`, then confirm `FUSE_WR_DONE=1` and `OWNER_PK_HASH_LOCK=1`.
6. Throughout steps 6–9: service `etrng_req` (§9), RW1C-clear non-fatal errors, and stop on fatal.

7. Poll `status[1]` (`ready_for_mb_processing`). Read `FW_BYTES` from fuse ctrl `0x10`.
8. Poll `MBOX_LOCK` until bit 0 = 0, and check `MBOX_USER = 1`. Write `MBOX_CMD=0x46574C44` and `MBOX_DLEN=253440`. Then for each of 63,360 words: read `soc_fw_store[i]` and write `MBOX_DATAIN`. Then write `MBOX_EXECUTE=1`.
9. Poll `MBOX_STATUS[3:0]` until non-zero, expect 2 (CMD_COMPLETE), then write `MBOX_EXECUTE=0`.
10. Poll `status[2]` (`ready_for_runtime`), then write magic `0xCA11F00D` (on the LEDs: `0x0D`).

The magic/LED codes the TB uses are 1 = start, 2 = ready_for_fuses, 3 = AxUSER programmed, 4 = fuses done, 5 = ready_for_mb_processing, 6 = FW_LOAD issued, 7 = mailbox complete, `0xCA11F00D` = runtime.

Note on clocks. The TB clock is 100 MHz (`#5`), but `TIMER_CONFIG=20000` ps assumes 50 MHz. This only affects Caliptra's MTIME scaling, not the boot result. Set `TIMER_CONFIG` to match the real board clock.

---

12. Changes vs. the 1.x design

Area	1.x (rev 3)	2.x (this design)
Bus	APB, 18-bit, PAUSER	AXI4, 19-bit, AxUSER (32-bit)
Bridge	`avmm_to_apb`	`avmm_to_axi`
Window	256 KiB (18-bit)	1 MiB (20-bit)
Requester IDs	One shared PAUSER = 1	Nios = 1, fuse ctrl = 2 (unique)
Fuses	83 words, data-only image	104 words + 12 owner hash + lock, address/data image, 117 entries
Owner PK hash	Fuse `FUSE_OWNER_PK_HASH`	Register `CPTRA_OWNER_PK_HASH[12]` + `_LOCK` (pwrgood)
Lifecycle	`FUSE_LIFE_CYCLE` + straps	Straps only
Ready signal	`ready_for_fw_push`	`ready_for_mb_processing`; both on `status_pio` bit 1, active-high
Mailbox	128 KiB	256 KiB
ROM / ICCM / DCCM	48 / 128 / 128 KiB	96 / 256 / 256 KiB
PQC	LMS optional	ML-DSA-87 (Adams-Bridge) or LMS, selected by `FUSE_PQC_KEY_TYPE`
New tie-offs	—	`strap_ss_*`, `ss_debug_intent`, `ss_ocp_lock_en`, obf UDS/FE, recovery, DMA AXI manager, `cptra_csr_hmac_key` (new strap)
FW bundle	109,000 B	253,440 B

---

13. Compliance checklist vs. caliptra-rtl v2.1.2 Integration Specification

References: IS Table 4 (clocks/resets), Table 5 (AXI), Tables 11–13 (straps, generic wires), §TRNG, Table 19 (SoC integration requirements), and `soc_ifc` RDL. The status reflects the **2.x RTL glue plus the TB host sequence**. The ported Nios firmware (`quartus_a5e_2x_nios/sw/app/main.c`) follows the same sequence but has not been executed in simulation or on hardware.

Requirement	Spec ref	Implementation	Status
Clock running before / ≥10 clk around `cptra_pwrgood`	Table 19 Resets and Clocks	Free-running clk; TB 20 clk pwrgood→rst_b	**PASS** (sim)
Async assert / sync deassert of pwrgood, rst_b	Table 4 / RDC	`reset_sync.sv` 2FF cells: `rst_b` 1→0 async, 0→1 sync; `pwrgood` 0→1 async, 1→0 sync (literal Table 4; the 0→1 source is a `clk` flop, §10.1)	**PASS**
Agilex Reset Release before user logic	Intel A5E	`rst_release` gates `sys_reset_n`	**PASS** (stub in sim)

Requirement	Spec ref	Implementation	Status
AXI single outstanding, in order, no narrow `mbox_datain`/`TRNG_DATA`	IS Unsupported features	`avmm_to_axi` single-beat, `WSTRB=F`	**PASS**
Drive AxUSER on all requests; never `0xFFFFFFFF`	Table 19 AXI USER	Nios 1, fuse 2	**PASS**
AxUSER unique per agent, non-zero	Table 19 AXI USER	1 vs 2	**PASS**
Mailbox AxUSER programmed + locked before use	Table 19 MAILBOX rules	Step 4, before any mailbox access	**PASS** (sim; ideally done by immutable SoC ROM)
TRNG AxUSER programmed + locked	Table 19 TRNG rules	Step 4	**PASS**
Fuse AxUSER locked or hardened	Table 19 FUSE rules	Programmed + locked to 2; `CPTRA_SET_FUSE_AXI_USER_INTEG=0`	**PASS** (programmable; hardening optional)
Fuse writes + `FUSE_WR_DONE` by immutable logic or ROM	Table 19 Fuses	`soc_fuse_ctrl2x` FSM, but the image is a rebuildable hex in FPGA RAM	**TEST-ONLY**
UDS/FE from OTP, no debug inject path	Table 19 Fuses	STUDY values in a hex image	**TEST-ONLY**
Debug read paths for fuses disabled in Production	Table 19 Fuses	Fuse range gated to user 2; UDS/FE write-only; but the hex image is readable in the bitstream	**TEST-ONLY**
Owner PK hash written + locked before mutable code	`soc_ifc_external_reg.rdl`	Fuse ctrl writes + locks (pwrgood-sticky); Nios could write it in the window before START	**PASS** (study); production: write from immutable logic before the host CPU can reach AXI
Warm reset: SoC re-writes `FUSE_WR_DONE` only	IS Fuses / Boot FSM	`START_DONE` implemented	**OPEN** (not simulated in 2.x)
Security state from the SoC's security state	Table 19 Security State	Hardwired Production + debug locked	**TEST-ONLY**
Obfuscation key: NIST-generated, not software, off scan, stable before reset	Table 19 Obfuscation Key	Fixed study localparam	**TEST-ONLY**
CSR HMAC key generated/rotated per project	Table 19 CSR HMAC Key	Fixed study constant (latched only in MANUFACTURING)	**TEST-ONLY**
Subsystem straps / `ss_*` / obf UDS/FE / recovery tied 0 in passive	Tables 11–12	Tied in `caliptra_passive_top`	**PASS**
`ss_ocp_lock_en` constant	Table 11	`1'b0`	**PASS**
DMA AXI manager not connected in passive	IS DMA / AXI arbitration	Ready/valid tied 0	**PASS**
`generic_input_wires` bit 0 = zeroization status; others 0	Table 13	`64'b0`	**PASS**
TRNG serviced by a hardware gasket (FIPS)	IS §TRNG	Host firmware/TB model, pseudo-random data	**TEST-ONLY**
Report Caliptra error outputs	Table 19 Error reporting	Polled on `status_pio[5:6]`; not a Nios IRQ	**OPEN** (IRQ)
Recover fatal only via pwrgood reset	Table 19 Error reporting	Planned: clear `ctrl[0]`/`[1]`, re-raise, full fuse push (the 1.x `main.c` does this with ≤3 retries); not in the 2.x TB	**OPEN**
SoC programs WDT before debug-locked ROM	HW spec WDT	`WDT_CFG` written before fuses; ROM log confirms	**PASS**
`TIMER_CONFIG` programmed	`soc_ifc` RDL	20000 ps (50 MHz)	**PASS** (TB clock is 100 MHz; set per board)
`scan_mode` 0 functional; 1 for 5,000 clk before scan	Table 19 DFT	Tied 0, no scan	**PASS** (FPGA)

Requirement	Spec ref	Implementation	Status
Caliptra TAP as a TAP endpoint	Table 19 DFT	Tied off	**OPEN**
Input clock from an on-die oscillator; clock-attack defence	Table 19 Resets and Clocks	Board clock	**OPEN**
No independent power-gating; clock gating via Caliptra FW	Table 19 Power saving	Clock ungated	**PASS**
SRAM zeroization on pwrgood	IS SRAM	Not implemented (bitstream-time zero only)	**OPEN**
Synthesis integrity (`u__size_only__` prims preserved)	IS Synthesis constraints	Not yet addressed in the Quartus flow	**OPEN**
Device fits	—	126 % ALMs on A5E 065	**FAIL** (§14)

Summary: 32 rows: 17 PASS, 7 OPEN, 7 TEST-ONLY, 1 FAIL (fit). Every TEST-ONLY item is intentional for a study and must be replaced for a product.

---

14. A5E 065 fit result and options

From `docs/caliptra-2x/P1_FIT_RESULT_2X_A5E.md`. Quartus Prime Pro 25.3.0 b109, Analysis & Synthesis only. The top is `caliptra2x/rtl_2x/caliptra_passive_top.sv` (Caliptra core only, **no Nios**). `MLDSA_MASKING` is on (upstream default).

Resource	2.x synth	A5EC065	%	1.x (Nios project†)
ALMs needed (pre-fit estimate)	**279,685**	**222,400**	**125.8 %**	138,715
Combinational ALUTs	395,094	—	—	218,761
Dedicated logic registers	411,569	889,600	46.3 %	141,933
Block memory bits	9,005,696	31.46 Mb	≈ 28.6 %	≈ 5.67 Mb
M20K (estimate)	≈ 484	1,611	≈ 30 %	—
DSP (synth / est. post-merge)	213 / 199	846	25.2 % / 23.5 %	49

† The 1.x column is the `quartus_a5e_nios/` project: Nios V plus Caliptra 1.x, synthesized with Quartus Prime Pro 25.3.0 b109 (Analysis & Synthesis only; source: `CALIPTRA_2X_INTEGRATION_PLAN.md` §3.1). Its scope differs by row:

- **ALMs and block memory:** the whole design (`nios_caliptra_a5e.syn.summary`: 138,715 ALMs).
- **ALUTs and registers:** the `caliptra_top` hierarchy row only.
- **DSP:** 49 in synthesis (47 post-merge).

So this column is **not** the same top as the 2.x column, which is the core-only `caliptra_passive_top`. For a like-for-like comparison, the 1.x core-only wrapper (`quartus_a5e/`, `caliptra_a5e.syn.summary`) estimates 141,256 / 222,400 ALMs (64 %), 142,373 registers and 47 DSP post-merge.

- **Biggest consumer: Adams-Bridge** (`abr_inst`): 101,640 ALUTs, 244,925 FFs, 164 DSP. The masked NTT `hybrid_bf2x2` alone has 218,303 FFs.
- **Next:** `ecc_top1` (79,991 ALUTs), VeeR (55,132), key vault (51,019).
- **Over by 57,285 ALMs.** Even a 20 % packing gain leaves the design at ≥ 100 %. With the Nios V system, fw store and glue added (`caliptra2x/quartus_a5e_2x_nios/`, §17) the full design estimates **286,415 / 222,400 ALMs (129 %)**.

Options

Option	Fits	Spec-conformant	Comment
A. Agilex 7 (e.g. AGF/AGI 014 ≈ 487k ALMs, ~57 % on this estimate; 022/027 for margin)	Yes	Yes	Unmodified `caliptra2x/rtl_2x/` + caliptra-rtl v2.1.2 RTL; change FAMILY/DEVICE in `caliptra2x/quartus_a5e_2x/caliptra2x_a5e.qsf` (and `gen_qsf.py` if you regenerate it); new PD/pins
B. Simulation-only 2.x (keep Caliptra 1.x in A5E hardware)	n/a	Yes	Develop the glue, firmware and tooling in Verilator / sw-emulator now

Option	Fits	Spec-conformant	Comment
C. Unmasked / reduced test build (`MLDSA_MASKING` undefined and/or `RV_FPGA_OPTIMIZE`)	Unknown, untested	**No**: removes side-channel countermeasures	Per the plan, upstream `MLDSA_MASKING` mainly zeroes the randomness; real savings may need an RTL fork. Research only

---

15. Open items

1. ~~Create the Platform Designer system `nios_caliptra2x_sys`~~ **Done** (`caliptra2x/quartus_a5e_2x_nios/qsys/`, §18).
2. ~~Port `main.c` to the §11 sequence~~ **Done** (`caliptra2x/quartus_a5e_2x_nios/sw/app/main.c`). Still to do: run it on a Nios RTL simulation or a board (only the TB behavioral host is simulated).
3. Add 2.x `warm` and `fatal` Verilator tests, as in the 1.x suite.
4. Choose the hardware target (Agilex 7 vs. sim-only), then do a full fit, timing, and synthesis-integrity constraints.
5. Production replacements for every TEST-ONLY row: OTP fuses, a real lifecycle, an HSM-generated obfuscation and CSR HMAC key, a hardware TRNG, and own signing keys (FIRMWARE_AND_KEYS_2X.md).
6. Export the bridge's Avalon `response` (`cl_response`, 2'b10 = SLVERR) as a `caliptra_axi_response` port of `nios_caliptra2x_sys`. Today it is left unconnected in `nios_caliptra2x_top.sv`, so the Nios cannot see AXI errors.

---

16. Getting the sources

The upstream Caliptra RTL is **not copied into this repo**. It comes in as pinned git submodules under `external/`, used unmodified. Every FPGA-specific change lives in this repo.

Path	What	Version / commit
`external/caliptra-rtl-1x/`	chipsalliance/caliptra-rtl (submodule)	**v1.1**, `5f85fb4bc95b753a2f7d042db7dc2644ca1e8c49`
`external/caliptra-rtl-2x/`	chipsalliance/caliptra-rtl (submodule)	**v2.1.2**, `49370266d12cb0c4a8f71b3a0ff7e54ba7d4866e`
`external/caliptra-rtl-2x/submodules/adams-bridge/`	chipsalliance/adams-bridge (nested submodule)	v2.0.3, `b77e3d899e828d626cfc2a0d26a6b5704cc121e0`
`rtl/`, `quartus/syn_src/`	1.x passive wrapper + synthesis overrides (`caliptra_sram`, `caliptra_top_tb_pkg`, `caliptra_veer_sram_export`)	this repo
`quartus_a5e_nios/`	Nios V + Caliptra 1.x Quartus project: `rtl/` glue, `qsys/` Platform Designer system (generated output committed), `ip/rst_release.ip`, `sw/` BSP + app, `onchip_mem.hex`	this repo
`quartus_a5e/`	Caliptra 1.x core-only Quartus project (size study)	this repo
`fw/`	1.x ROM (rom-1.1.1), signed FW bundle, fuse image	this repo
`sim/`	Verilator TB for the 1.x Nios glue	this repo
`caliptra2x/`	2.x glue (`rtl_2x/`), sim (`sim_2x/`, `sim/`), firmware (`fw_2x/`), fuse script (`scripts/`), core-only Quartus project (`quartus_a5e_2x/`), full Nios V + 2.x Quartus project (`quartus_a5e_2x_nios/`: `qsys/`, `sw/`, `ip/rst_release.ip`, `onchip_mem.hex`)	this repo
`scripts/get_caliptra_a5e.sh`	Clone or update this repo with all submodules, then verify the pins	this repo

Path	What	Version / commit
`scripts/fetch_deps.sh`	`git submodule update --init --recursive` (or clone the pinned commits if there is no `.git`), then verify; `--check` verifies only	this repo
`scripts/quartus_docker.sh`	Run any Quartus command in the Quartus Pro 25.3 Docker image with the repo and license mounted	this repo

All paths in the `.qsf`, `.f` and run scripts are **relative to the file's own directory**, so the checkout can be anywhere. If Quartus reports `caliptra_passive_top` or `caliptra_top` as undefined, the submodules are missing: run `scripts/fetch_deps.sh`.

Download (macOS or Linux; needs git; about 1 GB with both RTL trees):

```
# Option A: one script (clones, or updates an existing clone with pull + submodule update, then checks pins)
curl -fsSLO https://raw.githubusercontent.com/pwilliam282/caliptra-a5e-quartus/main/scripts/get_caliptra_a5e.sh
bash get_caliptra_a5e.sh ~/work/caliptra-a5e-quartus

# Option B: plain git
git clone --recursive https://github.com/pwilliam282/caliptra-a5e-quartus.git
cd caliptra-a5e-quartus
scripts/fetch_deps.sh            # expect: ok ... x3, "deps OK"
```

If you cloned without `--recursive`, or downloaded a ZIP, run `scripts/fetch_deps.sh` anyway.

---

17. Quartus build

Tools and host.

- Quartus Prime **Pro 25.3.0 Build 109** with Agilex 5 device support. All results in this guide come from that version.
- Quartus Pro runs only on **Linux or Windows x86-64**. On a Mac (including Apple-silicon MacBooks), use the Mac for git and editing, and run Quartus on a Linux x86-64 PC, VM or server. The Docker image is `linux/amd64` and is not usable under emulation on Apple-silicon Docker Desktop.
- Docker option: the official image `alterafpga/quartuspro-v25.3:agilex5`, wrapped by `scripts/quartus_docker.sh <dir> '<command>'`. The wrapper mounts the whole repo (including `external/`) at the same path, mounts your license read-only, runs as your UID, and uses `--net=host`.

License. The A5EC065 needs the no-cost **SW-AGILEX-5E** license from Altera's Self-Service Licensing Center. It is node-locked to the NIC MAC address of the machine that runs Quartus. In Docker this works because of `--net=host`: FlexLM reads the host NIC, and without it `quartus_syn` can segfault in the host-ID lookup. Point the tools at the file:

- Native: `export LM_LICENSE_FILE=/path/to/SW-AGILEX-5E.dat`
- Docker: `export LIC_HOST=/path/to/SW-AGILEX-5E.dat`, or put the file at `license/SW-AGILEX-5E.dat` in the repo (gitignored)

Never commit license `.dat` files.

Projects.

Project	Directory	`.qpf` / revision	Top
Nios V + Caliptra 1.x (the 1.1 design)	`quartus_a5e_nios/`	`nios_caliptra_a5e`	`nios_caliptra_top`
Nios V + Caliptra 2.x (full system; same Nios system as 1.1, §18; does not fit)	`caliptra2x/quartus_a5e_2x_nios/`	`nios_caliptra2x_a5e`	`nios_caliptra2x_top`
Caliptra 1.x core only (size study)	`quartus_a5e/`	`caliptra_a5e`	`caliptra_passive_top`
Caliptra 2.x core only (fit gate; 126 %, does not fit)	`caliptra2x/quartus_a5e_2x/`	`caliptra2x_a5e`	`caliptra_passive_top`

Step 1: generate IP (Nios project only). The Platform Designer output (`qsys/nios_caliptra_sys/`, `qsys/ip/nios_caliptra_sys/`) is committed. The Reset Release IP output (`ip/rst_release/`) is gitignored. Run `quartus_ipgenerate` once after cloning (and after any `.qsys`/`.ip` change). It regenerates every `QSYS_FILE`/`IP_FILE` in the project:

```
cd quartus_a5e_nios
quartus_ipgenerate nios_caliptra_a5e
# Optional, only if you changed the PD system: rebuild it from Tcl first
#   cd qsys && qsys-script --script=nios_caliptra_sys.tcl --quartus-project=none && cd ..
#   (or: qsys-generate qsys/nios_caliptra_sys.qsys --synthesis=VERILOG --quartus-project=nios_caliptra_a5e)
```

For the **2.x Nios project** it is the same, in `caliptra2x/quartus_a5e_2x_nios/` with revision `nios_caliptra2x_a5e` and Tcl `qsys/nios_caliptra2x_sys.tcl`; its program memory is `caliptra2x/quartus_a5e_2x_nios/onchip_mem.hex` (rebuild with `caliptra2x/quartus_a5e_2x_nios/sw/build_app.sh`), and `run_nios2x_a5e_docker.sh [syn|fit]` runs the whole flow.

`run_nios_a5e_docker.sh [syn|fit]` does the full regeneration (Tcl → PD → ipgenerate → synthesis) in one go. The Nios program memory is initialised from the committed `quartus_a5e_nios/onchip_mem.hex`. Rebuild it with `quartus_a5e_nios/sw/build_app.sh` (needs `RISCV_BIN=<dir with riscv-none-elf-gcc>`) only if you change `sw/app/main.c`.

Step 2: compile. Native Quartus on Linux:

```
cd quartus_a5e_nios
quartus_syn --analysis_and_elaboration nios_caliptra_a5e   # quick check: all files found, hierarchy elaborates
quartus_syn nios_caliptra_a5e                              # Analysis & Synthesis
quartus_fit nios_caliptra_a5e                              # place + route (needs >= 32 GB RAM)
quartus_sta nios_caliptra_a5e                              # timing
quartus_asm nios_caliptra_a5e                              # .sof (only with real pins and a license that enables
it)
# or all stages:  quartus_sh --flow compile nios_caliptra_a5e
```

Same thing with Docker, from the repo root:

```
export LIC_HOST=$HOME/licenses/SW-AGILEX-5E.dat
scripts/quartus_docker.sh quartus_a5e_nios 'quartus_ipgenerate nios_caliptra_a5e && quartus_syn nios_caliptra_a5e'
scripts/quartus_docker.sh quartus_a5e_nios 'quartus_fit nios_caliptra_a5e && quartus_sta nios_caliptra_a5e'
scripts/quartus_docker.sh quartus_a5e 'quartus_syn caliptra_a5e'
scripts/quartus_docker.sh caliptra2x/quartus_a5e_2x 'quartus_syn caliptra2x_a5e'
scripts/quartus_docker.sh caliptra2x/quartus_a5e_2x_nios 'quartus_ipgenerate nios_caliptra2x_a5e && quartus_syn
nios_caliptra2x_a5e'
```

GUI alternative.

1. *File → Open Project →* `quartus_a5e_nios/nios_caliptra_a5e.qpf`.
2. *Tools → Generate IP* (or accept the prompt to generate IP).
3. *Processing → Start → Start Analysis & Synthesis* (or *Start Compilation* for the full flow).

Outputs. Reports go to `<project>/output_files/`:

- `<rev>.syn.summary`: ALMs, registers, DSP
- `<rev>.syn.rpt`: hierarchy, resource usage by entity, messages
- `<rev>.syn.ae.rpt`: Analysis & Elaboration
- `<rev>.fit.*`, `<rev>.sta.*`, `<rev>.sof` after fit, timing and assembler

The `qdb/`, `dni/` and `tmp-clearbox/` work directories are gitignored.

Measured results (Quartus Pro 25.3.0 b109, A5EC065BB32AE6S, from a fresh `git clone --recursive` of commit `28951ad` on a 16 GB / 8-core Linux box, Docker image above):

Project	Analysis & Elaboration	Full Analysis & Synthesis
`quartus_a5e_nios` (`nios_caliptra_top`)	**OK**, 0 errors, 1 min 19 s, 3.3 GB peak; `quartus_ipgenerate` OK, 0 errors, 9 s	**Successful**, 0 errors: **138,715 / 222,400 ALMs (62 %)**, 145,784 registers, 47 DSP post-merge, 5,671,936 block-memory bits; 6 min 50 s at `--parallel=4`, 4.0 GB peak. This matches the earlier build exactly
`quartus_a5e` (`caliptra_passive_top` 1.x)	**OK**, 0 errors, 44 s, 2.8 GB peak	141,256 / 222,400 ALMs (**64 %**), 142,373 registers, 47 DSP (committed `syn.summary`; 6 min 30 s, 3.6 GB)
`caliptra2x_a5e` (`caliptra_passive_top` 2.x)	**OK**, 0 errors, 1 min 41 s, 4.4 GB peak	279,685 / 222,400 ALMs (**126 %, does not fit**), 411,569 registers, 199 DSP (committed report; 22 min, 6.8 GB)
`nios_caliptra2x_a5e` (`nios_caliptra2x_top`, Nios V + 2.x)	OK (part of the full run); `quartus_ipgenerate` OK, 0 errors	**Successful**, 0 errors, 17 warnings: **286,415 / 222,400 ALMs (129 %, does not fit)**, 413,568 registers, 199 DSP post-merge (213 before merge), 11,630,208 block-memory bits; 21 min at `--parallel=4`, 6.8 GB peak (2026-11-11). Of this, the Nios system `u_sys` is 3,948 ALUTs / 3,528 registers / 527,360 RAM bits, `u_fw_store` 2,097,152 bits (256 KiB), `u_fuse_ctrl` 235 ALUTs, `u_bridge` (`avmm_to_axi`) 17 ALUTs. `quartus_fit` was tried and **killed by the Linux OOM killer** about 3 min in (at the timing-setup step of the fitter plan stage) at 13.9 GB resident on the 15 GB box (2026-11-11, 09:50 CST), so no fitter report exists; with 129 % ALM use, fit would fail anyway

All three hierarchies elaborate with no missing-file or undefined-entity messages. The ROM, FW-bundle and fuse hex files are read by `$readmemh` from their repo-relative paths (`../fw/…`, `../fw_2x/…`).

Expected warnings (harmless):

- Critical Warning 16668: MFCU turns off auto-discovery. MFCU is intentional, because Caliptra relies on macros persisting across files.
- Critical Warning 20759 on the core-only projects: no Reset Release IP. That IP is only in the Nios top.
- Critical Warning 127004 (Nios project, ×2): Quartus pads the internal MIFs for the ROM (6144 → 8192 words) and the FW store (28672 → 32768 words) to a power of two. This is expected.
- Critical Warning 18061/18010: power-up-level notes on the security-state latch.
- Verilog truncation and parameter warnings from upstream RTL.

Known issues.

- **Intermittent Quartus start-up crash in Docker:** `Error (22912): Unhandled exception … DDM_T::verify_token(token) : Cannot identify the client` within the first second. It is not a design problem. Simply rerun the same command; on the build box it took up to 3 tries.
- **Synthesis hangs with no CPU use.** Quartus can stop progressing, usually during the SLD/JTAG-hub `ip-deploy` step or right after RAM inference when it starts its parallel helper processes. Signs: the log stops growing, `quartus_syn` sits at about 0 % CPU, and defunct `quartus_sh` children remain. Most likely the same start-up crash, hitting a helper process. Kill the container (`docker kill <id>`; with `DOCKER_NAME=<name>` set for `scripts/quartus_docker.sh` you can `docker kill <name>`) and rerun. A healthy Nios synthesis shows 4 busy `quartus_syn` helpers with `--parallel=4`. The run then finishes normally; the earlier note recorded about 7 min at `--parallel=8`.
- **Memory.** A&E needs about 3–4.5 GB, and full synthesis about 4 GB (1.x) or 7 GB (2.x). **Fit/route needs ≥ 32 GB**: placement passed on a 16 GB VM but routing ran out of memory, so fit, timing and `.sof` have not been produced yet.
- **Pins.** All top-level I/O except `clk` (and `reset_n` in the Nios project) are `VIRTUAL_PIN ON`. Assign real pins and I/O standards for your board before `quartus_asm`.

---

18. Nios V system: Caliptra 1.1 vs 2.x

The 2.x Nios host system is the Caliptra 1.1 system with **only the Caliptra window changed**. `caliptra2x/quartus_a5e_2x_nios/qsys/nios_caliptra2x_sys.tcl` was derived line-for-line from

`quartus_a5e_nios/qsys/nios_caliptra_sys.tcl`. After generation, the `.ip` files of `clk`, `ram`, `jtag_uart`, `ctrl_pio`, `status_pio` and `magic_pio` are **identical** to the 1.1 ones apart from the system name. `cpu.ip` differs only in the CPU's address-map string (the window entry), and `caliptra_win.ip` only in address width / span (18 → 20 bit, 256 KiB → 1 MiB). The HAL BSP was regenerated with `niosv-bsp -t=hal` from the 2.x system: every `settings.bsp` value is identical to 1.1, and `system.h` differs only in `ALT_SYSTEM_NAME` and `CALIPTRA_WIN_BASE`/`_SPAN`.

Same in both

Item	Caliptra 1.1 (`quartus_a5e_nios/`)	Caliptra 2.x (`caliptra2x/quartus_a5e_2x_nios/`)
PD system	`nios_caliptra_sys`	`nios_caliptra2x_sys`
CPU	`intel_niosv_m` (Nios V/m), reset vector `ram.s1` + 0, debug module on	same
Clock source	`clock_source` 50 MHz, reset sync deassert	same
Program/data RAM	`intel_onchip_memory` 64 KiB × 32, init `onchip_mem.hex`, at `0x0000_0000`	same
JTAG UART	`altera_avalon_jtag_uart` at `0x0008_0000`, IRQ 0	same
`ctrl_pio`	32-bit output, reset 0 (Caliptra held in reset), `0x0008_0010`	same
`status_pio`	32-bit input, `0x0008_0020`	same (bit 1 is `ready_for_mb_processing` instead of `ready_for_fw_push`)
`magic_pio`	32-bit output, reset 0, `0x0008_0030`; `led = magic[7:0]`	same
Timer / debug agents	`cpu.timer_sw_agent` `0x0008_0040`, `cpu.dm_agent` `0x0009_0000`	same
Reset	Reset Release IP `ip/rst_release.ip` gates `sys_reset_n`; 2FF async-assert/sync-deassert syncs	same file, same gating
SDC	50 MHz `clk`, `altera_reserved_tck`, false paths on `reset_n` and `u_rst_release`	same (`nios_caliptra2x.sdc`)
Device / pins	A5EC065BB32AE6S; all I/O virtual except `clk`, `reset_n` (`led[7:0]` virtual)	same
BSP	HAL, `niosv-bsp` defaults, stdio = `jtag_uart`, sys clk = `cpu`	same settings
Firmware flow	`sw/app/main.c`: cold release, user/lock + timer + WDT, fuse START, TRNG service, FW load, runtime; fatal pwrgood recovery (≤ 3), warm-reset helper	same flow, ported to 2.x registers
LED / MAGIC codes	1, 2, 3, 4, 5, 6, 7, `0xCA11F00D`; `0xDEAD000E/000F/00FF/0010`; warm `0x13/0x14/0x15/0x17`	same
Build	`run_nios_a5e_docker.sh [syn|fit]`, `sw/build_app.sh`	`run_nios2x_a5e_docker.sh [syn|fit]`, `sw/build_app.sh`

Different (because Caliptra 2.x is different)

Item	Caliptra 1.1	Caliptra 2.x
Window bridge (`caliptra_win`, `altera_avalon_mm_bridge`)	18-bit, 256 KiB at `0x0004_0000`, exported as `caliptra_apb`	20-bit, 1 MiB at **`0x0010_0000`**, exported as **`caliptra_axi`** (a 1 MiB window must be 1 MiB-aligned)
Bridge to Caliptra (top RTL)	`avmm_to_apb` → APB, PAUSER	`avmm_to_axi` → AXI4, AxUSER
Window map	fw store `0x00000`, fuse ctrl `0x1F000`, Caliptra APB `0x20000`–`0x3FFFF`	fw store `0x00000`, fuse ctrl `0x7F000`, Caliptra AXI `0x80000`–`0xFFFFF` (`mbox_csr` `+0x20000`, `soc_ifc` `+0x30000`)
Requester IDs	PAUSER 1 for Nios and fuse ctrl	Nios AxUSER 1, fuse ctrl AxUSER 2 (`FUSE_VALID_AXI_USER = 2`)
FW store / bundle	28,672 words, 109,000 B	65,536 words, 253,440 B
Fuse writes	84 (83 + `FUSE_WR_DONE`)	118 (104 + owner PK hash ×12 + lock + `FUSE_WR_DONE`)
Ready signal on `status_pio[1]`	`ready_for_fw_push`	`ready_for_mb_processing`

Item	Caliptra 1.1	Caliptra 2.x
`main.c` text size	26,300 B	26,504 B

How it was checked

- The exported ports of the generated `nios_caliptra2x_sys` (`qsys/nios_caliptra2x_sys/synth/nios_caliptra2x_sys.v`) are exactly the ones `nios_caliptra2x_top.sv` connects and the TB stand-in in `sim_2x/tb_nios2x_top_fw.sv` declares, so no change to the top or the TB was needed.
- `.ip` files diffed against 1.1 after renaming the system (results above).
- `main.c` uses the same register offsets as the passing TB host model (`tb_nios2x_top_fw.sv`) and the `caliptra_reg_defines.svh` values of v2.1.2 (§5.3).
- Not done: running `main.c` on an RTL model of the Nios (the Verilator TB uses the behavioral host), and running it on a board (the 2.x design does not fit the A5E 065).

`onchip_mem.hex` byte order (fixed for 1.1 too). Quartus `elf2hex -w 32` writes each 32-bit word MSB first. The fallback `scripts/elf2hex_intel.py` used to write the bytes in little-endian file order, so its output was byte-swapped relative to Quartus. The 1.1 `quartus_a5e_nios/onchip_mem.hex` in git had been made with that fallback. Both are fixed: the fallback now matches Quartus `elf2hex` byte-for-byte (checked on the 2.x image), and the 1.1 image was regenerated from the same, unchanged 1.1 `main.c`.

Caliptra 2.x + Nios glue — Simulation Guide

Where the files are. All paths in this guide are repo-relative (pwilliam282/caliptra-a5e-quartus):

- 2.x glue RTL, sim, firmware, fuse script and Quartus project: `caliptra2x/` (`caliptra2x/rtl_2x/`, `caliptra2x/sim_2x/`, `caliptra2x/sim/`, `caliptra2x/fw_2x/`, `caliptra2x/scripts/`, `caliptra2x/quartus_a5e_2x/`)
- Upstream RTL: `external/caliptra-rtl-2x/` = chipsalliance/caliptra-rtl v2.1.2 (`49370266`) with `submodules/adams-bridge` (`b77e3d8`), a pinned git submodule, unmodified. Fetch it with `scripts/get_caliptra_a5e.sh` or `git clone --recursive` + `scripts/fetch_deps.sh`. See NIOS_CALIPTRA_2X_INTEGRATION.md §16.
- Companion docs `P1_FIT_RESULT_2X_A5E.md` and `CALIPTRA_2X_INTEGRATION_PLAN.md`: this folder (`docs/caliptra-2x/`). The 1.x guide is `docs/NIOS_CALIPTRA_INTEGRATION.md`.

caliptra-sw is not vendored (only the built ROM/bundle in `caliptra2x/fw_2x/`).

- **Scope:** Verilator simulation of the **real** 2.x glue RTL (`caliptra2x/rtl_2x/`) around caliptra-rtl v2.1.2 (`49370266`) core-only. It runs the real ROM `rom-2.1.3-1` and the real FW bundle `fw-2.1.4` (FMC + runtime), and boots from cold power-on to `ready_for_runtime`.
- **Result: PASS**: `TB_NIOS2X PASS cyc=36617987 trng=217 (prod, debug locked)`.
- **Attestation and DPE:** `+test=attest` continues after `ready_for_runtime` with 34 runtime mailbox commands (DICE cert chain, `STASH_MEASUREMENT`, PCR quotes, DPE GetProfile/CertifyKey/GetCertificateChain): **PASS** at cycle 59,280,395, certificates byte-identical to the sw-emulator. See §8. What the Nios model covers: §8.1. Quick start: §8.2.
- **Related:** NIOS_CALIPTRA_2X_INTEGRATION.md (design), FIRMWARE_AND_KEYS_2X.md (ROM, bundle, fuse image). The raw record is in `caliptra2x/sim_2x/BOOT_NIOS2X.md` and `caliptra2x/sim_2x/nios2x_top_fw_run.log`.

---

1. What is simulated

Part	Real RTL or model
`nios_caliptra2x_top` (window decode, `soc_fw_store`, `soc_fuse_ctrl2x`, `avmm_to_axi`, AXI write mux, 2FF reset syncs, straps)	**Real** (`caliptra2x/rtl_2x/`)
`caliptra_passive_top` + `caliptra_top` v2.1.2 (VeeR, soc_ifc, crypto, Adams-Bridge, memories)	**Real** (`external/caliptra-rtl-2x/`, `caliptra2x/rtl_2x/caliptra_passive_top.sv`)
Caliptra ROM / FMC / runtime	**Real** binaries: `caliptra2x/fw_2x/caliptra-rom.hex`, `caliptra2x/fw_2x/caliptra-fw-bundle.hex`
Fuse image	**Real** generator output: `caliptra2x/fw_2x/fuse_image_2x.hex`
Agilex Reset Release IP	Stub `caliptra2x/sim/rst_release_stub.sv` (`ninit_done` = 1 for 25 ns)
Platform Designer system `nios_caliptra2x_sys` (Nios V + PIOs + window)	**Behavioral model** `module nios_caliptra2x_sys` in `tb_nios2x_top_fw.sv`. Ports: `clk_clk`, `reset_reset_n`, `caliptra_axi_address[19:0]`, `_read`, `_write`, `_writedata[31:0]`, `_byteenable[3:0]` (always `4'hF`), `_readdata[31:0]`, `_readdatavalid`, `_waitrequest`, `_burstcount`, `_debugaccess`, `ctrl_pio_export[31:0]` (out, initialized to 0), `status_pio_export[31:0]` (in), `magic_pio_export[31:0]` (out, initialized to 0). It issues the Avalon accesses the 2.x Nios firmware is meant to issue

The Nios V CPU, Platform Designer interconnect and C firmware are **not** in this sim. Compare the 1.x sim, which also used a behavioral host. §8.1 describes what the attestation run covers and what it does not, and the next step that would exercise the real Nios V.

---

2. Tools and inputs

Item	Value
Simulator	**Verilator 5.054** (2026-11-01, conda-forge build), on `PATH` (set `VERILATOR_BIN=<dir>` to prepend a specific install; the build box used a conda env). Verilator 5.040 also works with the fixes in §7
C++ compiler	Verilator's `--timing` support needs C++20 coroutines. GCC 13 or newer and clang work as is. **GCC 10–12 need `-fcoroutines`**: the run scripts call `scripts/verilator_cflags.sh`, which adds `-CFLAGS -fcoroutines` automatically when `${CXX:-c++}` is GCC 10–12. Override with `VERILATOR_CFLAGS="…"` (empty to pass nothing) or pick a compiler with `CXX=…`
Build flags (`run_nios2x_top_fw.sh`)	`--binary --threads ${THREADS:-4} -j 8 -O3 -Wno-fatal -Wno-lint -Wno-style -Wno-UNOPTFLAT -Wno-MULTIDRIVEN -Wno-TIMESCALEMOD --top-module tb_nios2x_top_fw -Mdir obj_nios2x -f boot2x.f -o Vtb_nios2x`
Caliptra RTL	`external/caliptra-rtl-2x/` = v2.1.2 `49370266d12cb0c4a8f71b3a0ff7e54ba7d4866e`, with submodule `adams-bridge b77e3d8`. Pinned git submodule; run `scripts/fetch_deps.sh` (or clone with `--recursive`)
File list	`caliptra2x/sim_2x/boot2x.f` (641 lines): `+incdir+` paths and 588 caliptra-rtl source files (`../../external/caliptra-rtl-2x/...`, relative to `caliptra2x/sim_2x/`), plus the repo files listed below
Repo sources in the file list	`caliptra2x/sim_2x/patches/decompress_top.sv`, `caliptra2x/rtl_2x/caliptra_veer_sram_export.sv`, `caliptra2x/rtl_2x/caliptra_passive_top.sv`, `caliptra2x/sim/rst_release_stub.sv`, `caliptra2x/rtl_2x/reset_sync.sv`, `caliptra2x/rtl_2x/avmm_to_axi.sv`, `caliptra2x/rtl_2x/soc_fuse_ctrl2x.sv`, `caliptra2x/rtl_2x/soc_fw_store.sv`, `caliptra2x/rtl_2x/nios_caliptra2x_top.sv`, `caliptra2x/sim_2x/tb_nios2x_top_fw.sv`
Verilator patch	`caliptra2x/sim_2x/patches/decompress_top.sv` replaces the adams-bridge original. It passes the `abr_piso_multi` rate arrays as indexed patterns with a default (`'{0:64, 1:64, 2:64, 3:64, default:0}` and `'{0:4, 1:20, 2:44, 3:48, default:0}`), so it elaborates whether a Verilator version sizes the array from `NUM_MODES` (4) or from the module default (5). Earlier 4-entry positional patterns failed on Verilator 5.040 with `Assignment pattern missed initializing elements: 4`; fixed in `bc3a381`
Data files (via RTL parameter defaults, relative to `caliptra2x/sim_2x/`)	`../fw_2x/caliptra-rom.hex`, `../fw_2x/caliptra-fw-bundle.hex`, `../fw_2x/fuse_image_2x.hex`

All paths in `caliptra2x/sim_2x/boot2x.f` are relative to `caliptra2x/sim_2x/` (the run script `cd`s there): repo files as `../rtl_2x/…`, upstream RTL as `../../external/caliptra-rtl-2x/…`. The checkout can live anywhere; no path rewriting is needed.

---

3. Build and run

Run long sims detached. A full boot takes about 1.4 h of wall time, and a foreground shell or agent session can be interrupted.

```
scripts/fetch_deps.sh            # once, from the repo root: external/caliptra-rtl-2x must be present
cd caliptra2x/sim_2x
# build (~6 min) + run, detached; output in nios2x_top_fw_run.log, wrapper output in run.out
nohup setsid ./run_nios2x_top_fw.sh +test=all > run.out 2>&1 &

# re-run without rebuilding
BUILD=0 nohup setsid ./run_nios2x_top_fw.sh +test=all > run.out 2>&1 &

# watch progress
tail -f nios2x_top_fw_run.log          # LOG lines, TRNG services, heartbeats, milestones
grep -E 'MILESTONE|PASS|FAIL|TIMEOUT' nios2x_top_fw_run.log
```

Env var	Default	Meaning
`BUILD`	`1`	`0` skips the Verilator build
`THREADS`	`4`	Verilator `--threads`

Env var	Default	Meaning
`VERILATOR_BIN`	unset	Directory prepended to `PATH` if set; otherwise `verilator` from `PATH`
`LOG`	`nios2x_top_fw_run.log`	Sim log file name (the attestation flow uses `LOG=attest_run.log`)

- The script runs the binary under `stdbuf -oL -eL`, so the log is line-buffered and `tail -f` works. When the run ends, it appends `wall_time_s=<s> end <date>`.
- **Measured costs** on this box:
  - Build: Verilator walltime **358 s** (elab 4.9 s, convert 204.8 s, C++ build 143.9 s); 2,185 MB allocated; 134 C++ files.
  - Run: **5,069 s wall (1 h 24 m 29 s)** in the wrapper. Verilator reported walltime 4,452.7 s, cpu 17,793.6 s on 4 threads, 18 MB allocated, about 82 µs of sim time per wall second.

3.1 Plusargs

Plusarg	Default	Effect
`+test=axiuser`	—	Cold release, `ready_for_fuses`, AxUSER programming, then the AxUSER checks (§4.1), then `$finish`. Takes a few hundred cycles
`+test=prod`	—	Cold boot to `ready_for_runtime` **without** the AxUSER checks
`+test=all`	**default**	AxUSER checks, then the prod cold boot. **Not** a multi-boot suite: there is no warm or fatal second boot, unlike 1.x `+test=all`
`+test=attest`	—	Prod cold boot, then the file-driven runtime mailbox loop for `attest/attest_driver.py` (§8)
`+attest_dir=DIR`	`attest_out/rtl`	Folder for the `req_NNN.hex` / `resp_NNN.hex` protocol files (`+test=attest`)
`+max_cycles=N`	300,000,000	Prints `TB_NIOS2X TIMEOUT magic=… trng=…` and stops
`+hb=N`	1,000,000	Heartbeat line period: `[cyc] heartbeat status=<pio[7:0]> trng=<n>`

The TB clock is 100 MHz (`always #5`), so 1 cycle = 10 ns of sim time. `$finish` at 366 ms equals 36.6 M cycles.

---

4. What the testbench checks

4.1 AxUSER test (`test_axiuser`, run by `axiuser` and `all`)

This runs after the host programs and locks the AxUSERs (fuse = 2, TRNG = 1, mailbox[0] = 1). The TB `forces dut.nios_axi_user` and reads the bridge's `err_q` (SLVERR).

Access	AxUSER	Expected	Result (log)
Write `MBOX_LOCK`	3	SLVERR	`[115] MBOX write AxUSER=3 -> SLVERR OK`
Write `FUSE_UDS_SEED[0]`	3	SLVERR	`[121] fuse write AxUSER=3 -> SLVERR OK`
Write `FUSE_UDS_SEED[0]`	1 (Nios)	SLVERR (only user 2 may touch fuses)	`[127] … SLVERR OK`
Read `CPTRA_HW_REV_ID`	1	Accepted	`[132] AxUSER=1 HW_REV_ID=00000212 accepted OK → TB_AXIUSER PASS`

4.2 Boot checks (`prod`, `all`)

Check	FAIL message if it fails
`generic_input_wires === 0`	`TB_NIOS2X FAIL generic_input_wires`
Fuse ctrl ID = `0x46555345`	`FAIL fuse id`
Fuse ctrl ends DONE, not ERR	`FAIL fuse ctrl ERR (SLVERR on a fuse write?)`
Fuse ctrl `COUNT == 118` (104 fuse + 13 owner hash/lock + WR_DONE)	`FAIL expected COUNT=118 …`

Check	FAIL message if it fails
`CPTRA_FUSE_WR_DONE[0] == 1`	`FAIL FUSE_WR_DONE not set`
`CPTRA_OWNER_PK_HASH_LOCK[0] == 1`	`FAIL owner PK hash not locked`
`MBOX_USER == 1` after the lock is acquired	`FAIL MBOX_USER`
`MBOX_STATUS == 2` (CMD_COMPLETE) after FIRMWARE_LOAD	`FAIL mbox status` (plus a register dump)
No `cptra_error_fatal` at any time	`FAIL unexpected cptra_error_fatal` (plus a dump)
`etrng_req` drops within 100,000 cycles after each `DATA_WR_DONE`	`FAIL etrng_req stuck …`
`ready_for_runtime` before `max_cycles`	`TIMEOUT`

The TB also **prints, without checking**: `HW_REV_ID`, `HW_CONFIG`, `CPTRA_SECURITY_STATE`, `FUSE_PQC_KEY_TYPE` readback, `CPTRA_OWNER_PK_HASH[0]`, and a final dump of BOOT/FLOW status and all four error registers.

PASS criteria: every check above holds, `ready_for_runtime` is reached, and then 20,000 more cycles run with no fatal error. The TB prints `SUMMARY …` and `TB_NIOS2X PASS cyc=<n> trng=<n> (prod, debug locked)`.

4.3 The `FUSE_PQC_KEY_TYPE readback=00000000` quirk

- The image writes `FUSE_PQC_KEY_TYPE = 1` at `0x3038C` (line 91 of the 117-line image, written by the fuse ctrl with AxUSER 2). The TB reads it back **as the Nios (AxUSER 1)** and gets `00000000`.
- **Cause:** in caliptra-rtl `soc_ifc_arb.sv`, `soc_reg_req` for addresses in `SOC_IFC_FUSE_START_ADDR..SOC_IFC_FUSE_END_ADDR` (`0x30200–0x305FF`) requires `valid_fuse_user`. After `CPTRA_FUSE_AXI_USER_LOCK` = 1, that means AxUSER == 2. The Nios read therefore never reaches the register block, and the read data is 0.
- The same `valid_fuse_user` gate rejects the AxUSER 1 and AxUSER 3 fuse *writes* with SLVERR in §4.1. The TB does not check the bridge error flag on this read, so whether that read also returned SLVERR was not recorded.
- **Proof the fuse was effective:** in Production lifecycle, the ROM compares the manifest PQC type with this fuse and would fail with `IMAGE_VERIFIER_ERR_PQC_KEY_TYPE_MISMATCH`. Instead it logged `[fwproc] Img verified ecc=0 MLDSA=0 svn=0 fuse_svn=0` at cycle 17,010,257.
- By contrast, `CPTRA_OWNER_PK_HASH*` (`0x30140`/`0x30170`) and `CPTRA_FUSE_WR_DONE` (`0x300B0`) are outside the fuse range, so the Nios read them back correctly.
- To read the fuse values back for debugging, issue the reads with AxUSER 2 (for example, add a read path to `soc_fuse_ctrl2x`) or before `CPTRA_FUSE_AXI_USER_LOCK` is set.

4.4 Log channel

- The ROM, FMC and runtime print through `generic_output_wires[7:0]`, with bit 8 toggling per character.
- The TB top assembles lines and prints them as `[cycle] LOG <text>`.
- It records the first `[fmc` / `[alias rt` line as `rom_to_fmc` and the first `[rt]` line as `fmc_to_rt`.

---

5. Measured results (`+test=all`, PID 227528, 2026-11-10)

Environment:

- Start 13:53:45 CST, end 15:18:14 CST.
- `HW_REV_ID = 00000212`.
- `HW_CONFIG = 00000010` (`iTRNG_en=0`, `SUBSYSTEM_MODE_en=0`).
- `CPTRA_SECURITY_STATE = 00000007` (lifecycle 3 = Production, debug_locked = 1).

Cycle	Event (from `nios2x_top_fw_run.log`)
40	`MILESTONE ready_for_fuses`
132	`TB_AXIUSER PASS`
136	`FUSE_CTRL NUM_FUSES=117` (START follows)

Cycle	Event (from `nios2x_top_fw_run.log`)
614	`FUSE_CTRL STATUS=00000002 COUNT=118`
619	`FUSE_WR_DONE=00000001`; `FUSE_PQC_KEY_TYPE readback=00000000` (§4.3); `OWNER_PK_HASH_LOCK=00000001`; `OWNER_PK_HASH[0]=48afdb07`
634	`MILESTONE fuses done via soc_fuse_ctrl2x (COUNT=118)`
378,838	`Running Caliptra ROM ...`
379,157	First TRNG request serviced
384,577	`[state] Starting the WD Timer 100000000 cycles`
442,177 / 445,304	`LifecycleState = Production` / `DebugLocked = Yes`
794,349	`ROM Digest: AB0AEDDBE4F660DC3CDB6F3221DCE2ADA1F6D0593A403B0B12D9EB5CC5C632EB`
5,524,388	`[kat] --` (ROM KATs done)
5,542,288	`MILESTONE ready_for_mb_processing trng=21`
6,049,774	`MILESTONE FIRMWARE_LOAD issued, 253440 bytes from soc_fw_store`
14,288,568	`[fwproc] Recv command 0x46574c44`
17,010,257	`[fwproc] Img verified ecc=0 MLDSA=0 svn=0 fuse_svn=0`
17,085,903	`Load FMC at address 0x40000000 len 30760`
17,304,364	`Load Runtime at address 0x40008000 len 205516`
21,319,080	`MILESTONE MBOX_STATUS=2 (2=CMD_COMPLETE)`
27,820,645	`[exit] Launching FMC @ 0x40000000`
27,875,426	`Running Caliptra FMC ...`
30,035,513	First FMC log line (ROM → FMC)
36,003,133	`Caliptra RT` (RT KATs incl. MLKEM1024 follow)
36,217,560	First RT log line `[rt] OCP LOCK Unsupported` (FMC → RT)
36,596,602	`[rt] Disable entropy source`
36,597,923	`[rt] RT listening for mailbox commands...`
36,597,957	**`MILESTONE ready_for_runtime`**
36,617,987	`TB_NIOS2X PASS cyc=36617987 trng=217 (prod, debug locked)`

Summary line: `SUMMARY ready_for_fuses=40 fuses_done=614 first_trng=379157 ready_for_mb_processing=5542288 fw_load_issued=6049774 mbox_complete=21319080 rom_to_fmc=30035513 fmc_to_rt=36217560 ready_for_runtime=36597957 trng_fills=217 fuse_writes=118`

Final dump: `CPTRA_BOOT_STATUS=00000000`, `CPTRA_FLOW_STATUS=b8000000`, `HW_ERROR_FATAL = FW_ERROR_FATAL = HW_ERROR_NON_FATAL = FW_ERROR_NON_FATAL = 00000000`.

Not covered by this run: warm reset (START_DONE), fatal-error pwrgood recovery, mailbox commands after runtime (see §8 for the `+test=attest` run that covers them), the real Nios V / PD interconnect, and the A5E Reset Release IP model.

---

6. Waveforms

The 2.x TB does not dump waveforms as built. `boot2x.f` / `run_nios2x_top_fw.sh` do not pass `--trace`/`--trace-fst` (the build log shows `-DVM_TRACE=0 -DVM_TRACE_FST=0`), and `tb_nios2x_top_fw.sv` has no `$dumpfile`/`$dumpvars`.

To get waves (untested for 2.x; a full 36.6 M-cycle dump of the whole design would be very large and much slower):

1. Add `--trace-fst --trace-structs` (optionally `--trace-depth 3`) to the `verilator` line in `run_nios2x_top_fw.sh`.
2. In `module tb_nios2x_top_fw`, add a windowed dump so only the interesting part is recorded, for example around the fuse push:

```
initial begin
  $dumpfile("nios2x.fst");
  $dumpvars(2, tb_nios2x_top_fw.dut);   // glue level; raise depth for Caliptra internals
  #20us $dumpoff;                         // fuses + start of ROM; move/extend as needed
end
```

3. Rebuild (`BUILD=1`) and run with a smaller `+max_cycles` if you only need the beginning.
4. View with GTKWave: `gtkwave caliptra2x/sim_2x/nios2x.fst`. Useful signals:
   - `dut.status_in[7:0]`, `dut.ctrl_out[2:0]`, `dut.cptra_pwrgood`, `dut.cptra_rst_b`
   - `dut.fuse_busy`, `dut.u_fuse_ctrl.st`, `dut.u_fuse_ctrl.idx`
   - `dut.awaddr`, `dut.awuser`, `dut.awvalid`/`awready`, `dut.bresp`, `dut.u_bridge.st`, `dut.u_bridge.err_q`
   - `dut.etrng_req`, `dut.generic_output_wires[8:0]`

The 1.x project's waveform flow (FST + GTKWave screenshots) is in the 1.x repo history; this guide makes no claim that it has been redone for 2.x.

---

7. Troubleshooting

Symptom	Likely cause
`FAIL fuse ctrl ERR` / `COUNT` < 118	A fuse write got SLVERR: fuse AxUSER not programmed/locked to 2 before START, or a bad address in the image
`FAIL etrng_req stuck`	TRNG AxUSER not 1/locked, or `DATA_WR_DONE` (2) not written
ROM fatal around image verification	Fuse image doesn't match the bundle: regenerate with `caliptra2x/scripts/gen_fuse_image_2x.py` after every bundle rebuild (FIRMWARE_AND_KEYS_2X.md §5)
Mailbox command rejected / `0x01020009`	Mailbox used with the reserved AxUSER `0xFFFFFFFF` or an unlocked user (see the integration plan)
Build: `coroutine header requires -fcoroutines`, `'coroutine_handle' in namespace 'std' does not name a template type` (in `verilated_timing.h`)	Host GCC is 10–12 and the flag was not added (Verilator calls a different compiler than `${CXX:-c++}`). Run with `VERILATOR_CFLAGS=-fcoroutines ./run_nios2x_top_fw.sh` …, or `CXX=/path/to/g++-13`. Then `rm -rf obj_nios2x` and rebuild
Build: `Vtb_nios2x_top_fw__pch.h.fast` (or `.slow`): `linker input file not found` on every `.o`	The site Verilator was configured with an empty `CFG_CXXFLAGS_PCH_I`, so the precompiled header is passed as a bare file instead of `-include`. `scripts/verilator_cflags.sh` now adds `-MAKEFLAGS CFG_CXXFLAGS_PCH_I=-include` for GCC (disable with `VERILATOR_PCH_FIX=0`). `git pull`, `rm -rf obj_nios2x`, rebuild
Link: `undefined reference to 'pthread_create'` / `pthread_setaffinity_np` in `verilated_threads.o`	Site Verilator configured without `CFG_LDLIBS_THREADS`. `scripts/verilator_cflags.sh` now adds `-CFLAGS -pthread -LDFLAGS -pthread -LDFLAGS -lpthread` (disable with `VERILATOR_PTHREAD_FIX=0`). `git pull`, `rm -rf obj_nios2x`, rebuild
Build: `decompress_top.sv:84: Assignment pattern missed initializing elements: 4`	Old copy of `patches/decompress_top.sv` from before `bc3a381`; `git pull` and rebuild
`TIMEOUT` with no progress	Check heartbeats (`status=` bits) and `trng=` count; the run is long (≈ 36.6 M cycles)

---

8. ROM boot, firmware load, attestation and DPE in simulation

This flow runs the same cold boot as §3 (real ROM `rom-2.1.3-1`, real FW bundle `fw-2.1.4`, production straps, debug locked), then keeps the runtime alive and acts as the Nios host: it sends **34 runtime mailbox commands** that dump the full DICE certificate chain, stash a Nios measurement, quote the PCRs and drive DPE. It is the `+test=attest` mode of `tb_nios2x_top_fw.sv`. The detailed record (command payloads, certificate decode, measurement path) is `caliptra2x/sim_2x/ATTEST_2X.md`.

Reference result (RTL, Verilator 5.054, 2026-11-10): `TB_NIOS2X PASS cyc=59280395 trng=379 (prod, debug locked)`. All 34 commands returned `DATA_READY` with `fwerr=00000000` and a valid checksum. Every certificate is **byte-identical** to the caliptra-sw sw-emulator run, and both the ECC P-384 and ML-DSA-87 chains verify strictly with `openssl verify` up to a TEST manufacturing CA.

8.1 What this simulation covers (Nios model)

The design under the Nios is the **real** RTL you would compile in Quartus: `caliptra2x/rtl_2x/nios_caliptra2x_top.sv`. That includes:

- the window decode
- `soc_fw_store`, which holds the 253,440-byte firmware bundle
- the `soc_fuse_ctrl2x` fuse controller
- the `avmm_to_axi` bridge, the AXI write-channel mux, and the 2FF reset synchronizers
- `caliptra_passive_top`
- unmodified Caliptra 2.x **v2.1.2** RTL, running the real ROM **2.1.3** and the FMC and runtime firmware **2.1.4**

The Platform Designer system `nios_caliptra2x_sys` — the Nios V CPU, its on-chip RAM, the JTAG UART and the PIOs — is **not** simulated as RTL. In its place is a behavioral SystemVerilog module of the same name and ports, in `tb_nios2x_top_fw.sv`. It drives the Avalon window directly with read and write tasks, following the same register sequence as the Nios firmware:

- AxUSER, timer and watchdog setup
- fuse programming through the fuse controller
- TRNG service
- `FIRMWARE_LOAD` over the mailbox
- the 34 runtime mailbox commands, which come from `attest/attest_driver.py`

So the bus traffic Caliptra sees matches what the firmware should produce, but no Nios instructions execute. This run does **not** cover Nios V instruction execution, the 2.x `caliptra2x/quartus_a5e_2x_nios/sw/app/main.c` or `onchip_mem.hex`, or the JTAG UART. TRNG data is testbench `$urandom` (not a real entropy source). All keys are TEST/STUDY values. DPE Sign is not exercised.

Reference result. `TB_NIOS2X PASS` at 59,280,395 cycles. All 34 mailbox commands returned status OK (`DATA_READY`, `fwerr=0`). There were no fatal or non-fatal errors. All 17 certificates were byte-identical to the caliptra-sw emulator. The three `QUOTE_PCRS` digest/signature values differ only because the emulator hashes the nonce in the opposite word order (see §8.8); that is not an RTL failure.

Milestone	Cycle
Fuses done	611
`ready_for_mb_processing`	5,542,265
`FIRMWARE_LOAD` issued	6,049,751
Firmware load complete	21,319,057
ROM → FMC	30,035,490
FMC → RT	36,217,537
`ready_for_runtime`	36,597,934

The later mailbox-command cycles are in the §8.4 table.

To close the gap and exercise the real Nios V and the 2.x `onchip_mem.hex`, the next step is a co-simulation that uses the Platform Designer simulation files for `nios_caliptra2x_sys` in place of the behavioral module. That will likely need Questa rather than Verilator, because the Nios V simulation model is typically encrypted.

8.2 Quick start: run it yourself

Prerequisites. Verilator 5.x (5.040 is fine), GCC, Python 3 and OpenSSL 3 (ideally ≥ 3.5, for ML-DSA-87) on `PATH`. On GCC 10–12 (for example SLES15 with GCC 12.2), set `export VERILATOR_CFLAGS=-fcoroutines`. The run script also applies the PCH and `-pthread` fixes in `scripts/verilator_cflags.sh` automatically (§7). You need a clone with the `external/caliptra-rtl-2x` submodule fetched.

```
# 1. Get the repo with its submodules (pinned 2.x RTL).
git clone --recursive https://github.com/pwilliam282/caliptra-a5e-quartus.git
cd caliptra-a5e-quartus
# If you already have a clone:
#   git pull && git submodule update --init --recursive

# 2. Start the sim. The first run builds first (a few minutes).
cd caliptra2x/sim_2x
rm -rf attest_out/rtl && mkdir -p attest_out/rtl
export VERILATOR_CFLAGS=-fcoroutines
LOG=attest_run.log nohup setsid ./run_nios2x_top_fw.sh +test=attest +max_cycles=4000000000 > attest_run.out 2>&1 &
# Optional: VERILATOR_BIN=/path/to/verilator/bin  (if verilator is not on PATH)
#           THREADS=8                               (if you have the cores)
#           BUILD=0                                 (skip the rebuild)

# 3. Start the mailbox driver in the same folder, right after the sim.
nohup setsid python3 attest/attest_driver.py attest_out/rtl > attest_out/rtl_driver.log 2>&1 &

# 4. Watch the progress.
grep -E 'MILESTONE|ATTEST (phase|resp|done)|PASS|FAIL|TIMEOUT' attest_run.log
# Expect the fuse / ROM / firmware-load / FMC / runtime milestones, then 34
# 'ATTEST resp ... status=1' lines, ending with TB_NIOS2X PASS at about
# 59.3 M cycles (~8–9 h wall time on a typical 4-thread box).

# 5. Check the result (after TB_NIOS2X PASS).
python3 attest/analyze.py attest_out/rtl attest_ref/emu
# Expect ANALYZE PASS, 17 matching certificates, and one expected
# QUOTE_PCRS raw-response warning (the emulator nonce-order bug, §8.8).
```

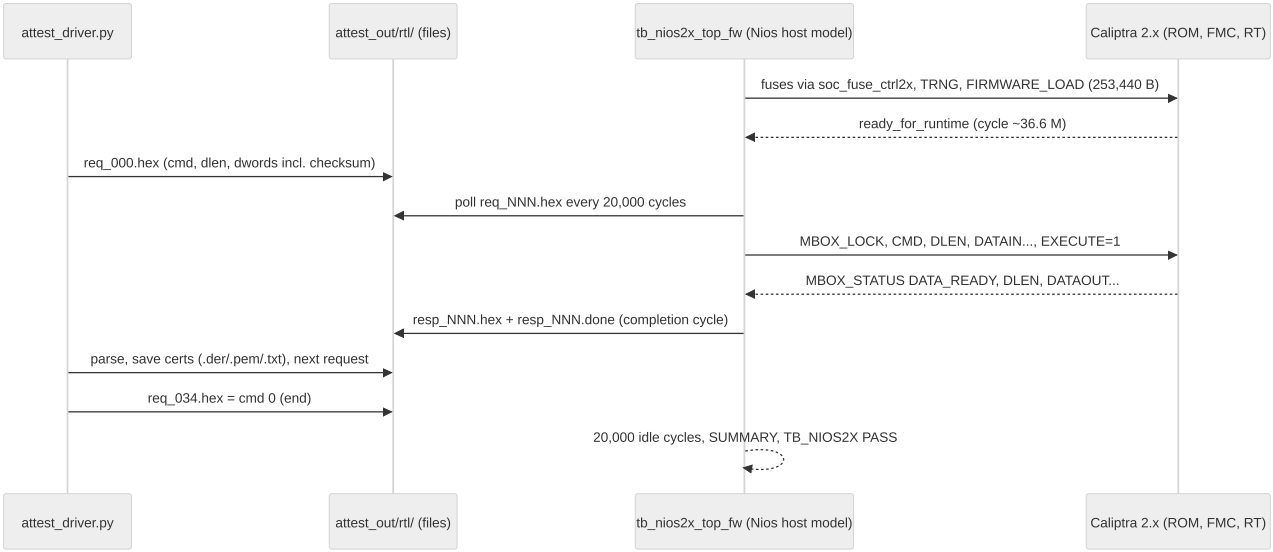
Certificates and raw responses land in `attest_out/rtl/`. The default `+max_cycles` (300 M) covers the 59.3 M-cycle run, but the TB keeps clocking while it waits for the driver, so use `4000000000` so a late or slow driver never trips `TIMEOUT`. Keep the `attest_out/rtl` path the same for the TB (`+attest_dir=…`, default `attest_out/rtl`) and the driver.

Troubleshooting (short). Build errors: look in `nios2x_build.log` (and §7 / §8.9). If the driver stops with a timeout (it waits at most six hours for any single response, for example if the machine slept), just rerun step 3; the sim picks up where it left off. More symptoms are in §8.9.

Other inputs already in the clone.

Item	Notes
Sources	`git clone --recursive` (or `git submodule update --init --recursive`) fetches `external/caliptra-rtl-2x`. You can also run `scripts/fetch_deps.sh` or `scripts/get_caliptra_a5e.sh` from the repo root
ROM / FW bundle / fuses	Prebuilt and committed: `caliptra2x/fw_2x/caliptra-rom.hex`, `caliptra-fw-bundle.hex`, `fuse_image_2x.hex` (checksums in `caliptra2x/fw_2x/SHA256SUMS`). To rebuild them, see FIRMWARE_AND_KEYS_2X.md. If you rebuild, the certificates change and will not match `attest_ref/`
TEST mfg CA	Committed in `caliptra2x/sim_2x/attest/testca/`: CA certificates plus the IDevID TBS/certs cached by IDevID public key. With the committed fuses **no private key is needed**. The CA `*.key` files are never committed; see §8.8 to make a new local CA

8.3 How it works



The Nios host in this diagram is the behavioral model described in §8.1.

- The testbench and the Python driver talk through files in `+attest_dir` (default `attest_out/rtl`, relative to `caliptra2x/sim_2x/`). The driver builds every request (header checksum `0 − Σ bytes`, DPE packets, IDevID TBS) and decodes every response. The TB only does the Nios register sequence, so the same driver also runs against the sw-emulator (`attest/emu/attest_emu.rs`).
- Start the driver together with the sim. It waits for `resp_NNN.done` (up to 6 h per command), and it deletes stale `req_*`/`resp_*` files from an earlier run when it starts.
- The TB keeps servicing `etrng_req` (TRNG) and checking `cptra_error_fatal` while it waits.

8.4 What each phase does and what to expect in the log

Cycle numbers are from the reference run (`attest_ref/rtl_tb_run.log`); a rerun should match within a few hundred cycles (TRNG timing). The boot milestones through `ready_for_runtime` are also summarized in §8.1.

Phase	What happens	Expected log line(s)	Cycle
Power-on, fuses	Cold release; Nios programs and locks AxUSERs (fuse = 2, TRNG = 1, mailbox = 1); `soc_fuse_ctrl2x` writes 117 fuse words + owner PK hash/lock + `FUSE_WR_DONE`	`MILESTONE ready_for_fuses`, `FUSE_CTRL STATUS=00000002 COUNT=118`, `MILESTONE fuses done via soc_fuse_ctrl2x ( COUNT=118)`	40 / 611
ROM boot	ROM starts, takes TRNG from the host, checks lifecycle (Production, debug locked), runs KATs, derives IDevID/LDevID (ECC + ML-DSA)	`LOG Running Caliptra ROM ...`, `TRNG request 1 serviced`, `LOG LifecycleState = Production`, `MILESTONE ready_for_mb_processing trng=21`	5,542,265
FIRMWARE_LOAD	Nios streams the 253,440-byte bundle from `soc_fw_store` into the mailbox (`FWLD`); ROM verifies the manifest (vendor ECC+ML-DSA keys, owner key, PQC type, SVN) and loads FMC + RT	`MILESTONE FIRMWARE_LOAD issued, 253440 bytes`, `LOG [fwproc] Img verified ecc=0 MLDSA=0 …`, `MILESTONE MBOX_STATUS=2 (2=CMD_COMPLETE)`	6,049,751 / 21,319,057
ROM → FMC	ROM builds the LDevID→FMC Alias certs and jumps to FMC	`MILESTONE first FMC log line (ROM -> FMC handoff)`	30,035,490
FMC → RT	FMC derives the RT Alias, launches the runtime; RT runs KATs, initializes DPE (default context)	`MILESTONE first RT log line (FMC -> RT handoff)`, `LOG [rt] RT listening for mailbox commands...`	36,217,537

Phase	What happens	Expected log line(s)	Cycle
Runtime ready	`ready_for_runtime`; the TB enters the attestation loop	`MILESTONE ready_for_runtime`, `=== ATTEST phase: waiting for requests in attest_out/rtl ===`	36,597,934
Firmware info, identity keys	`FW_INFO`, `GET_IDEV_ECC384_INFO`, `GET_IDEV_MLDSA87_INFO`	`ATTEST req 0 cmd=494e464f ('INFO')` … `ATTEST resp 2 status=1 dlen=2600`	36,624,580 – 36,739,042
Cert chain	`GET_LDEV_*_CERT`, `GET_FMC_ALIAS_*_CERT`, `GET_RT_ALIAS_*_CERT` (ECC P-384 and ML-DSA-87)	`ATTEST req 3 … ('LDEV')` … `ATTEST resp 8`	36,789,582 – 37,387,350
PCR quote (before)	`QUOTE_PCRS_ECC384` with nonce 1; PCR31 = 0	`ATTEST req 9 … ('PCRQ')`	38,327,090
DPE (ECC)	`INVOKE_DPE_ECC384` **GetProfile** (0x01), **CertifyKey** (0x09, default context, X.509)	`ATTEST req 10/11 … ('DPEC')`	38,375,977 / 42,659,941
Measurement	`STASH_MEASUREMENT` of the Nios image digest M (tag `NIOS`, svn 1): extends PCR31 and runs a DPE **DeriveContext** (make-default, locality = AxUSER 1) inside Caliptra	`ATTEST req 12 … ('MEAS')`, `LOG [rt] Received command=0x4d454153 (MEAS)`	42,720,549
PCR quote + DPE (after)	`QUOTE_PCRS_ECC384` nonce 2 (PCR31 = SHA-384($0^{48} \| M$)); `CertifyKey` again (leaf now has TCB `NIOS`); **GetCertificateChain** (0x10) ×2 (2,586 B)	`ATTEST req 13 … 16`	43,660,293 – 48,097,509
DPE (ML-DSA)	`INVOKE_DPE_MLDSA87` GetProfile, CertifyKey, GetCertificateChain ×12 (23,658 B), `QUOTE_PCRS_MLDSA87`	`ATTEST req 17 … ('DPEM')` … `req 31 … ('PCRM')`	48,765,037 – 58,932,857
IDevID certs	Driver signs the IDevID TBS with the TEST mfg CA; `GET_IDEV_ECC384_CERT` / `GET_IDEV_MLDSA87_CERT` assemble the certs	`ATTEST req 32 … ('IDEC')`, `req 33 … ('IDMC')`	58,991,424 / 59,240,360
End	Driver writes cmd 0; TB idles 20,000 cycles, dumps status and error registers (all 0)	`ATTEST done, 34 commands`, `SUMMARY …`, **`TB_NIOS2X PASS cyc=59280395 trng=379 (prod, debug locked)`**	59,280,395

The longest commands are the two ECC `CertifyKey` calls (≈ 4.3 M cycles each) and ML-DSA `CertifyKey` (≈ 1.3 M cycles). The driver log shows each command as `[driver] -> status=1 len=… fwerr=00000000 chk=True`, ending with `[driver] done, 34 commands`.

DPE commands exercised: GetProfile, CertifyKey (before and after the stash), GetCertificateChain, plus DeriveContext through `STASH_MEASUREMENT`. InitializeContext is done by the runtime at boot (default context). DPE **Sign**, an explicit DeriveContext/RotateContext/DestroyContext, `CERTIFY_KEY_EXTENDED` and exported CDIs are **not** issued by this testbench.

8.5 Outputs

Path (under `caliptra2x/sim_2x/`)	Content
`attest_run.log`	Sim log: milestones, ROM/FMC/RT `LOG` lines, `ATTEST req/resp` per command, `SUMMARY`, PASS line, Verilator stats, `wall_time_s`
`attest_out/rtl/req_NNN.hex`, `resp_NNN.hex`, `resp_NNN.done`	Raw protocol files (`.done` = completion cycle)
`attest_out/rtl/resp_NNN_<name>.bin`	Raw response bytes per command
`attest_out/rtl/{idevid,ldevid,fmc_alias,rt_alias}_{ecc384,mldsa87}.{der,pem,txt}`	DICE certificate chain (both algorithms)
`attest_out/rtl/dpe_{ecc,mldsa}_leaf_*.{der,pem,txt}`, `dpe_*_chain_N.*`, `dpe_*_cert_chain.bin`	DPE leaves and the DPE GetCertificateChain output
`attest_out/rtl/attest_summary.json`	Measurement, nonces, quotes (PCRs, digest, signature), per-command log
`attest_out/rtl/analysis.json`	Written by `analyze.py`: decoded DICE extensions, verification, compare
`attest_ref/{rtl,emu}/`	Committed reference results (RTL run and sw-emulator run)

`attest_out/`, `attest_run.log` and `attest_run.out` are git-ignored.

8.6 Verify the certificate chain

`analyze.py` does all of the following and prints `ANALYZE PASS` (exit 0) or `ANALYZE FAIL: …` (exit 1). To check by hand:

```
cd caliptra2x/sim_2x/attest_out/rtl
CA=../../attest/testca
for alg in ecc384:ecc mldsa87:mldsa; do a=${alg%%:*}; c=${alg##*:}
  dpe=dpe_${c}_leaf_after.pem
  # each link
  openssl verify -partial_chain -CAfile $CA/test_mfg_ca_$c.pem idevid_$a.pem
  openssl verify -partial_chain -CAfile idevid_$a.pem    ldevid_$a.pem
  openssl verify -partial_chain -CAfile ldevid_$a.pem    fmc_alias_$a.pem
  openssl verify -partial_chain -CAfile fmc_alias_$a.pem rt_alias_$a.pem
  openssl verify -partial_chain -CAfile rt_alias_$a.pem  $dpe
  # full chain, strict: TEST CA > IDevID > LDevID > FMC Alias > RT Alias > DPE leaf
  cat idevid_$a.pem ldevid_$a.pem fmc_alias_$a.pem rt_alias_$a.pem > /tmp/untrusted_$c.pem
  openssl verify -CAfile $CA/test_mfg_ca_$c.pem -untrusted /tmp/untrusted_$c.pem $dpe
done
# DICE extensions (TcbInfo / MultiTcbInfo): the post-stash DPE leaf has a TCB entry type 'NIOS', svn 1, fwid = M
openssl x509 -in dpe_ecc_leaf_after.pem -noout -text | less
# compare with the emulator / reference run
for f in *.der; do cmp -s $f ../../attest_ref/emu/$f && echo "same $f" || echo "DIFF $f"; done
```

Expected: every `openssl verify` prints `OK`, and all 17 `.der` files are `same`. `analyze.py` also checks that the DPE GetCertificateChain certs equal the mailbox certs (3/3 per algorithm), the IDevID cert key equals `GET_IDEV_*_INFO`, PCR31 = SHA-384(0^{48}‖M) after the stash, M appears only in the post-stash DPE leaves, and all quote signatures verify with the **FMC Alias** key.

8.7 Runtime and reference numbers

	sw-emulator (Step A)	RTL (Step B, reference)
Wall time	≈ 25 s	Verilator walltime 7,518 s (cpu 29,963 s on 4 threads, 78.9 µs sim/s); wrapper `wall_time_s=10639` (5:16:55 PM – 8:14:14 PM CST)
Build	—	≈ 6 min
Cycles	—	`ready_for_runtime` 36,597,934; first command done 36,624,580; last 59,240,360; `$finish` 59,280,395 (593 ms sim)
TRNG fills	—	379

The reference run was ≈ 3 h wall time on the development box (wrapper `wall_time_s=10639`). Plan for **≈ 8–9 h** on a typical 4-thread machine; `ready_for_runtime` is a bit over half of that.

8.8 Open items and limits

- **QUOTE_PCRS digest vs emulator (resolved: emulator nonce order).** The RTL and emulator return identical PCRs, nonce and reset counters, but different `digest`/signature. Cause: the RTL hashes the 8 nonce dwords in reverse order (`pv_gen_hash.sv` line 169, `nonce[NONCE_LEN_DWORD-1-nonce_offset_i]`, i.e. NONCE[7] first), while the caliptra-sw emulator hashes NONCE[0] first. With the RTL order, all three RTL quote digests (2× ECC, 1× ML-DSA) recompute exactly and verify with the FMC Alias key. `analyze.py` checks both orders. **A verifier for Agilex hardware quotes must use the RTL order**; one validated only against the emulator will reject real quotes. The 3 `resp_*QUOTE_PCRS*.bin` files are still listed as raw-response differences vs the emulator (expected); this is not a failure.
- **IDevID certificate** is assembled from a TBS signed by a local **TEST** mfg CA (Caliptra holds only the key). A product IDevID comes from the manufacturer's CA, usually via the Manufacturing-lifecycle CSR flow (not exercised).
- **New TEST CA.** Only needed if the IDevID key changes (other UDS/FE fuses): `ATTEST_CA_DIR=$PWD/attest_out/testca python3 attest/attest_driver.py attest_out/rtl` creates `test_mfg_ca_{ecc,mldsa}.{key,pem}` with `openssl genpkey` (P-384 / ML-DSA-87) and signs new TBSs; pass the same `ATTEST_CA_DIR` to `analyze.py`. Never commit the `*.key` files (git-ignored).

- The TB TRNG model returns `$urandom` words; key derivation is deterministic and signing uses deterministic nonces (ECDSA HMAC-DRBG, ML-DSA `sign_rnd = 0`), which is why the certificates match the emulator byte-for-byte.
- The Nios V CPU, 2.x `main.c` / `onchip_mem.hex` and JTAG UART are not in this sim (behavioral Avalon host only). A co-sim with the real Platform Designer Nios V model would close that gap; it will likely need Questa (§8.1).
- Not exercised: warm reset, ROM-time stash, `EXTEND_PCR`, DPE Sign, exported CDIs, waveforms (too long to dump).

8.9 Troubleshooting

Symptom	Likely cause / fix
Build errors (`-fcoroutines`, `__pch.h.fast`, `pthread_create`)	Same as §7: `VERILATOR_CFLAGS=-fcoroutines`, `git pull`, `rm -rf obj_nios2x`, rebuild
Sim sits at `=== ATTEST phase: waiting …` with only heartbeats	Driver not running or writing to another folder. Start `attest_driver.py attest_out/rtl` from `caliptra2x/sim_2x/` (or pass the same path with `+attest_dir=`)
Driver times out after six hours on one response	The driver waits at most 6 h per response. If the machine slept, or the sim was slower than expected, just rerun the driver (step 3 in §8.2); the sim resumes from the next request
Driver log stays at `req 000 FW_INFO` for over an hour	Normal: the runtime is only ready at ≈ 36.6 M cycles. Check `MILESTONE` lines in `attest_run.log`
`TB_NIOS2X TIMEOUT`	Raise `+max_cycles` (use `4000000000`)
`ATTEST resp N status=3` or `fwerr` ≠ 0	Command rejected. Check the `LOG [rt]` lines just before it; mailbox AxUSER must be 1 and locked; the request checksum must be correct (driver computes it)
Driver: `IDevID key not in the TBS cache …` `TEST CA private key … not available`	UDS/FE or ROM/bundle changed, so the IDevID key changed. Use a new local TEST CA with `ATTEST_CA_DIR` (§8.8); certificates will no longer match `attest_ref/`
`openssl: … ML-DSA-87 … unsupported` or ML-DSA verifies fail	OpenSSL < 3.5 on `PATH`. Install OpenSSL ≥ 3.5 and put it first on `PATH`
`analyze.py`: `ANALYZE FAIL: DER mismatch`	You rebuilt the ROM/bundle/fuses, or changed straps. Compare against your own emulator run instead of `attest_ref/emu`
`ANALYZE WARN … digest != … recomputation` (3×)	Expected: emulator hashes the nonce in the opposite word order (§8.8); not a failure
Second run replays old answers	Use `rm -rf attest_out/rtl` before starting (the driver also clears stale `req_*`/`resp_*` at start)

Caliptra 2.x — Firmware, Keys and Fuse-Image Preparation

Where the files are. All paths in this guide are repo-relative (pwilliam282/caliptra-a5e-quartus):

- 2.x glue RTL, sim, firmware, fuse script and Quartus project: `caliptra2x/` (`caliptra2x/rtl_2x/`, `caliptra2x/sim_2x/`, `caliptra2x/sim/`, `caliptra2x/fw_2x/`, `caliptra2x/scripts/`, `caliptra2x/quartus_a5e_2x/`)
- Upstream RTL: `external/caliptra-rtl-2x/` = chipsalliance/caliptra-rtl v2.1.2 (`49370266`) with `submodules/adams-bridge` (`b77e3d8`), a pinned git submodule, unmodified. Fetch it with `scripts/get_caliptra_a5e.sh` or `git clone --recursive` + `scripts/fetch_deps.sh`. See NIOS_CALIPTRA_2X_INTEGRATION.md §16.
- Companion docs `P1_FIT_RESULT_2X_A5E.md` and `CALIPTRA_2X_INTEGRATION_PLAN.md`: this folder (`docs/caliptra-2x/`). The 1.x guide is `docs/NIOS_CALIPTRA_INTEGRATION.md`.

caliptra-sw is not vendored (only the built ROM/bundle in `caliptra2x/fw_2x/`).

- **Applies to:** `caliptra2x/fw_2x/` and `caliptra2x/scripts/gen_fuse_image_2x.py` in this repo.
- **Source:** caliptra-sw `dd1e79d5bf55a9bd5bdb05915757eaf34844145d`, tags `rom-2.1.3-1` and `fw-2.1.4` (not vendored; clone `https://github.com/chipsalliance/caliptra-sw` at that commit and point `SW=` at it).
- **Hardware revision:** `--hw-revision 2.1`, matching caliptra-rtl v2.1.2 (`CPTRA_HW_REV_ID` reads `0x212`).
- **Related:** NIOS_CALIPTRA_2X_INTEGRATION.md §8 (fuse map), SIMULATION_2X.md.

> ⚠️ **The firmware bundle in `caliptra2x/fw_2x/` is signed with caliptra-sw's public "fake" test keys.** The private halves of every vendor and owner key are published in caliptra-sw `image/fake-keys/src/lib.rs`, for example `VENDOR_ECC_KEY_0_PRIVATE`, `VENDOR_MLDSA_KEY_0_PRIVATE`, `OWNER_ECC_KEY_PRIVATE` and `OWNER_MLDSA_KEY_PRIVATE`. A device fused with the hashes in `caliptra2x/fw_2x/fuse_image_2x.hex` will accept **any** firmware anyone signs with those keys. Use them for simulation and FPGA study only. §6 describes the production procedure.

---

1. Artifacts in `caliptra2x/fw_2x/`

File	Size	What it is
`caliptra-rom.bin`	98,304 B (96 KiB)	2.x ROM **with UART logging** (`ROM_WITH_UART`). SHA-256 `26d505a7ec938d14adbc82627cebd00e3240f7fdd3da464bb4a325c82e18cef2`
`caliptra-rom.hex`	12,288 lines	ROM as little-endian **64-bit** words, one per line (`$readmemh` into `caliptra_rom_m20k`)
`caliptra-fw-bundle.bin`	253,440 B	Signed bundle: manifest + FMC + runtime. SHA-256 `361bdf44d0267dc2f3073f62dc888c94ef929c6095e5d62a615c278b520524f2`
`caliptra-fw-bundle.hex`	63,360 lines	Bundle as little-endian **32-bit** words (`$readmemh` into `soc_fw_store`)
`SHA256SUMS`		`sha256sum *.bin` (covers the two `.bin` files only)
`fuse_image_2x.hex`	117 lines	Fuse image derived from the bundle (§5)
`build_rom.sh`		Rebuild script (§2)

Verify with `cd fw_2x && sha256sum -c SHA256SUMS`. Both hex files were checked word-for-word against the `.bin` files (all 12,288 / 63,360 words match).

---

2. Building the ROM and bundle (`caliptra2x/fw_2x/build_rom.sh`)

```
source ~/.cargo/env
SW=${SW:?}                                  # required: caliptra-sw checkout at dd1e79d
OUT=caliptra2x/fw_2x                        # build_rom.sh writes next to itself
cd $SW
cargo run --release --manifest-path builder/Cargo.toml --bin image -- --hw-revision 2.1 \
  --rom-with-log $OUT/caliptra-rom.bin --fw $OUT/caliptra-fw-bundle.bin
# then: pad + convert to caliptra-rom.hex (64-bit LE words) and caliptra-fw-bundle.hex (32-bit LE words)
sha256sum *.bin > SHA256SUMS
```

The `image` binary is `builder/bin/image_gen.rs` (`[[bin]] name = "image"` in `builder/Cargo.toml`). With the options above it does the following:

Option	Effect in `image_gen.rs`
`--hw-revision 2.1`	Sets `CALIPTRA_HW_REV=2.1` before building
`--rom-with-log FILE`	`build_firmware_rom(&firmware::ROM_WITH_UART)` (ROM with UART log → the `LOG` lines in the sim)
`--fw FILE`	`build_and_sign_image(&FMC_WITH_UART, &APP_WITH_UART_OCP_LOCK, image_options)`
(no `--image-options`)	`ImageOptions { fmc_version: version::get_fmc_version(), app_version: version::get_runtime_version(), fw_svn (default 0), pqc_key_type (default MLDSA), .. Default::default() }`

`ImageOptions::default()` (`builder/src/lib.rs`) supplies the keys:

Field	Default	Meaning
`vendor_config`	`caliptra_image_fake_keys::VENDOR_CONFIG_KEY_0`	4 vendor ECC P-384 keys, 4 ML-DSA-87 keys (32 LMS keys also defined); active `ecc_key_idx = 0`, `pqc_key_idx = 0`; **private keys included**; `pl0_pauser = 0x1`
`owner_config`	`Some(caliptra_image_fake_keys::OWNER_CONFIG)`	1 owner ECC P-384 key + 1 ML-DSA-87 key (+ LMS), **private keys included**
`pqc_key_type`	`FwVerificationPqcKeyType::MLDSA` (= 1; LMS = 3)	
`fw_svn`	0	

Values decoded from the bundle we built (parsed from `caliptra-fw-bundle.bin`):

Field	Value
Manifest marker / size	`0x324E4D43` ("CMN2") / 16,956 B
`pqc_key_type`	1 (ML-DSA)
ECC key descriptor	version 1, `key_hash_count` 4
PQC key descriptor	version 1, key type 1, `key_hash_count` 4
Active vendor key index (ECC / PQC)	0 / 0
Header `flags` / `toc_len` / `pl0_pauser` / `svn`	`0x1` / 2 / `0x1` / 0
FMC TOC	version `0x1044`, load = entry = `0x40000000`, offset 16,956, size 30,760
Runtime TOC	version `0x2010004`, load = entry = `0x40008000`, offset 47,716, size 205,516

The ROM log in the sim agrees: `Load FMC at address 0x40000000 len 30760` and `Load Runtime at address 0x40008000 len 205516`.

---

3. Bundle / manifest layout (2.x, "CMN2")

From caliptra-sw `image/types/src/lib.rs`. Offsets are into `caliptra-fw-bundle.bin`.

Offset	Size	Field
0	4	`marker` = `0x324E4D43`
4	4	`size` = 16,956
8	1 + 3	`pqc_key_type`, reserved
12	**1,736**	`preamble.vendor_pub_key_info` = ECC descriptor (4 + 4×48) + PQC descriptor (4 + 32×48) → **hashed into `FUSE_VENDOR_PK_HASH`**
1,748	4	`vendor_ecc_pub_key_idx`
1,752	96	`vendor_ecc_active_pub_key` (x, y)
1,848	4	`vendor_pqc_pub_key_idx`
1,852	2,592	`vendor_pqc_active_pub_key`
4,444	96 + 4,628	`vendor_sigs` (ECC r‖s, PQC signature)
9,168	**2,688**	`owner_pub_keys` = ECC (96) + PQC (2,592) → **hashed into `CPTRA_OWNER_PK_HASH`**
11,856	96 + 4,628	`owner_sigs`
16,580	8	reserved
16,588	160	`header` (revision, key idx, flags, toc_len, pl0_pauser, toc_digest, **svn @ +76**, vendor_data, owner_data @ +120)
16,748	2 × 104	TOC: FMC, runtime
16,956	30,760	FMC image
47,716	205,516	Runtime image
253,232	208	padding to the 253,440-byte bundle size

What gets signed (`image/gen/src/generator.rs`):

- **Vendor:** ECC-P384 signs SHA-384(`header[0..120]`), i.e. the header up to `owner_data` (bundle bytes 16,588–16,707). ML-DSA-87 signs those raw 120 bytes (pure ML-DSA, empty context).
- **Owner:** ECC signs SHA-384(the full 160-byte header, bytes 16,588–16,747). ML-DSA signs those raw bytes.
- The header's `toc_digest` covers the TOC, and each TOC entry's digest covers its image.
- Keys and signatures are stored in "hw format": 32-bit words, little-endian in the file, most-significant word first.
- *Verified:* the vendor and owner ECC signatures in our bundle check out against the active public keys over exactly these byte ranges. The ML-DSA ranges are taken from the generator source and were not independently verified.

---

4. Byte-to-hex conversion

`build_rom.sh` (inline Python):

- **ROM:** pad to a multiple of 8, then write each 8 bytes as `%016x` of a little-endian u64. That gives 12,288 lines for `CALIPTRA_IMEM_DEPTH` = 12288 × 64-bit.
- **Bundle:** pad to a multiple of 4, then write each 4 bytes as `%08x` of a little-endian u32. That gives 63,360 lines. `soc_fw_store` has 65,536 words, and the Nios streams words 0..63,359 into `MBOX_DATAIN`.

---

5. Fuse image generation (`caliptra2x/scripts/gen_fuse_image_2x.py`)

```
python3 caliptra2x/scripts/gen_fuse_image_2x.py [path/to/caliptra-fw-bundle.bin]   # default
caliptra2x/fw_2x/caliptra-fw-bundle.bin
# -> caliptra2x/fw_2x/fuse_image_2x.hex: 117 entries (104 fuse words + 13 owner-hash/lock); +FUSE_WR_DONE = 118 AXI
writes; pqc_key_type=1
```

What the script does:

1. **Checks the bundle.** It asserts:
   - marker = `CMN2`;
   - `pqc_key_type` ∈ {1, 3};
   - `manifest.size` = 16,956 (the 2.x layout);
   - header SVN = 0 (it refuses otherwise, because the SVN fuses are written as 0);
   - the PQC descriptor's key type equals the manifest `pqc_key_type`.
2. **Vendor hash.** `FUSE_VENDOR_PK_HASH` = SHA-384(bundle[12 : 12+1736]), written as 12 **big-endian** 32-bit words. This matches caliptra-sw `image_pk_desc_hash` / `bytes_to_be_words_48`, and the `caliptra-image-app --print-hashes` computation (`image/app/src/create/mod.rs`).
3. **Owner hash.** `CPTRA_OWNER_PK_HASH` = SHA-384(bundle[9168 : 9168+2688]) (owner ECC key 96 B + PQC key field 2,592 B), written as 12 **big-endian** 32-bit words to `0x30140–0x3016C`. Then `CPTRA_OWNER_PK_HASH_LOCK` = 1.
4. **PQC type.** `FUSE_PQC_KEY_TYPE` = manifest `pqc_key_type` (1).
5. **STUDY seeds.** STUDY UDS (16 words) and FE (8 words) from fixed formulas, **not secrets**. Every other fuse word is 0, except `FUSE_SOC_STEPPING_ID` = 1.
6. **Output.** Writes `{addr,data}` lines (see the integration guide §8 for the full table) and prints both hashes. With the current bundle:
   - `vendor_pk_hash 30399676 a17e3e97 3677b3ff 862f4bf2 d1932d88 4778453c 376fe00d c93fb8aa 0770f3eb f3411a08 53e9c57e ce8a2980`
   - `owner_pk_hash 48afdb07 3c5e0d4e e4649046 8ef81f2c f57249b6 e76a28f5 fca4de69 6a7d3e2e d3efc4e6 77431854 3e95307a 54988bd7`

Because the hashes come from the bundle itself, **re-run the script after every bundle rebuild or re-sign**, then rebuild the sim or bitstream (the image is `$readmemh`'d at elaboration). Re-running it on the committed bundle reproduces `fuse_image_2x.hex` byte for byte.

What the ROM checks against these values (`image/verify/src/verifier.rs`, when not Unprovisioned):

- `FUSE_PQC_KEY_TYPE` must equal the manifest PQC type (`IMAGE_VERIFIER_ERR_PQC_KEY_TYPE_MISMATCH`).
- `FUSE_VENDOR_PK_HASH` must be non-zero and match the descriptor digest.
- The active vendor key's hash must appear in the descriptor, and the key must not be revoked.
- The owner key digest must match `CPTRA_OWNER_PK_HASH`.
- Signatures must verify.
- Image SVN must be ≥ fuse SVN (unless anti-rollback is disabled).

---

6. Production key preparation (procedure)

Steps marked *(untested here)* follow the caliptra-sw source but were not run in this project.

6.1 Decide the key set

Key	Algorithm	Count	Held by
Vendor firmware-signing keys	ECC P-384	1–4 (`VENDOR_ECC_MAX_KEY_COUNT` = 4)	Silicon or firmware vendor HSM
Vendor PQC keys	**ML-DSA-87** (`pqc_key_type` = 1) **or** LMS (`= 3`)	ML-DSA 1–4 (`VENDOR_MLDSA_MAX_KEY_COUNT`), LMS 1–32 (`VENDOR_LMS_MAX_KEY_COUNT`)	Vendor HSM
Owner key	ECC P-384 + one PQC key of the bundle's single `pqc_key_type` (ML-DSA-87 if 1, LMS if 3)	1 each	Platform owner HSM

- All vendor public keys are hashed into one descriptor, so `FUSE_VENDOR_PK_HASH` commits to the whole set.
- Keep spare keys for revocation. The **last** key index can never be revoked (`verify_vendor_ecc_pk_idx`: "Last key index is never revoked").

6.2 Generate keys in an HSM

- Generate every private key **inside the HSM** (non-exportable). Export only the public keys.
- Formats the caliptra-sw tools read (`image/gen/src/lib.rs`, `image/crypto/src/openssl.rs`):

Key	Format accepted
ECC P-384 public	PEM `PUBLIC KEY` (SPKI)
ECC P-384 private (only for local test signing)	PEM, read via OpenSSL `EcKey::private_key_from_pem`
ML-DSA-87 public	Raw 2,592-byte file, **or** PEM SPKI (the BIT STRING payload is used)
ML-DSA-87 private (local test only)	Raw 4,896-byte expanded key, or PEM PKCS#8 whose privateKey octets are exactly that raw key
LMS public / private	PEM or raw `ImageLmsPublicKey` / `ImageLmsPrivKey` structs

- **Local test keys** with OpenSSL 3.5 (for a dry run of the flow, never for a product):

```
mkdir -p keys && chmod 700 keys            # keys/ is gitignored (see 6.6)
openssl ecparam -name secp384r1 -genkey -noout -out keys/vnd_ecc0_priv.pem
openssl ec -in keys/vnd_ecc0_priv.pem -pubout -out keys/vnd_ecc0_pub.pem
openssl genpkey -algorithm ML-DSA-87 -provparam ml-dsa.output_formats=priv-only -out keys/vnd_mldsa0_priv.pem
openssl pkey -in keys/vnd_mldsa0_priv.pem -pubout -out keys/vnd_mldsa0_pub.pem   # SPKI, accepted as-is
```

- **ML-DSA private key format caveat.** OpenSSL's PKCS#8 for ML-DSA wraps the 4,896-byte key in an inner OCTET STRING: the privateKey field is 4,900 bytes. This was checked with OpenSSL 3.5.7 on the box. The caliptra-sw reader takes those octets directly, so it would get 4,900 bytes and fail. Extract the raw key first, e.g. `python3 -c "import base64,sys;d=base64.b64decode(''.join(l for l in open(sys.argv[1]) if not l.startswith('-----')));open(sys.argv[2],'wb').write(d[-4896:])" keys/vnd_mldsa0_priv.pem keys/vnd_mldsa0_priv.bin` *(untested with the image tool)*.

6.3 Sign the firmware — mechanisms that exist in caliptra-sw `dd1e79d`

A. `caliptra-image-app create` with a key-config TOML (`image/app`).

- This signs **ELF** FMC/runtime with key files listed in a TOML. Arguments:
  - `--pqc-key-type 1|3`, `--key-config <toml>`, `--ecc-pk-idx`, `--pqc-pk-idx`
  - `--fmc <elf> --fmc-version N --fmc-rev <hex>`, `--rt <elf> --rt-version N --rt-rev <hex>`
  - `--fw-svn N --out <bundle>`, optional `--own-from-date/--own-to-date/--mfg-from-date/--mfg-to-date`, `--print-hashes`
- Key-config schema (`image/app/src/create/config.rs`); paths are relative to the TOML file:

```
[vendor]
ecc_pub_keys   = ["vnd_ecc0_pub.pem", "vnd_ecc1_pub.pem", "vnd_ecc2_pub.pem", "vnd_ecc3_pub.pem"]
lms_pub_keys   = []
mldsa_pub_keys = ["vnd_mldsa0_pub.pem", "vnd_mldsa1_pub.pem", "vnd_mldsa2_pub.pem", "vnd_mldsa3_pub.pem"]
ecc_priv_keys   = ["vnd_ecc0_priv.pem", ...]     # optional
mldsa_priv_keys = ["vnd_mldsa0_priv.bin", ...]   # optional
[owner]
ecc_pub_key   = "own_ecc_pub.pem"
lms_pub_key   = ""                               # field is required by the schema; read only for LMS
mldsa_pub_key = "own_mldsa_pub.pem"
ecc_priv_key   = "own_ecc_priv.pem"              # optional
mldsa_priv_key = "own_mldsa_priv.bin"            # optional
```

- If the private keys are omitted, the image is generated with **zero signatures**. The tool needs the private keys locally to sign, so with non-exportable HSM keys use **B** for the signatures.
- Get the ELFs with `image --all_elfs <dir>` (`builder/bin/image_gen.rs`).
- `--fmc-rev`/`--rt-rev`/`--pqc-pk-idx` are declared optional but are unwrapped in `run_cmd`, so pass them.

B. `image` builder with `--image-options`, `--hashes`, and signature override (`builder/bin/image_gen.rs`). This is the external/HSM path.

- `--image-options <toml>` replaces the `ImageOptions` struct (versions, `fw_svn`, `pqc_key_type`, full `vendor_config` / `owner_config` with inline public keys and optional private keys). The reference is `builder/test_data/default_image_options.toml`, which is a copy of the fake-key defaults, *including private keys*. For production, put your public keys in and **leave out** `priv_keys`.
- `--hashes <json>` writes `{"vendor": SHA-384(header[..owner_data]), "owner": SHA-384(header)}`. These are the ECC message digests "for external signing tools".
- `--vendor-sig-override <file>` / `--owner-sig-override <file>` replace `preamble.vendor_sigs` / `owner_sigs` with the bytes of `<file>`, parsed as `ImageSignatures`. That is the ECC signature (r‖s, 96 B, hw format) followed by the PQC signature field (4,628 B), 4,724 B total. The CLI help text still says "ECC … concatenated with an LMS signature"; the struct holds whichever PQC type is selected.

HSM signing procedure (B), (untested here):

1. Build once with your public-key-only `--image-options` plus `--hashes h.json`. The bundle comes out with zero signatures.
2. Sign with the HSM:
   - **ECC:** sign `h.json.vendor` with the active vendor ECC key and `h.json.owner` with the owner ECC key (ECDSA P-384 over the given SHA-384 digest).
   - **ML-DSA-87:** sign the **raw header bytes**, not the digest. Vendor = bundle[16588:16708], owner = bundle[16588:16748] (§3). Pure ML-DSA, empty context. `--hashes` does not export these bytes, so cut them from the step-1 bundle.
3. Assemble the two 4,724-byte `ImageSignatures` files. Convert r, s and the ML-DSA signature into caliptra's word order (§3); the ML-DSA signature is 4,627 B plus 1 pad byte.
4. Rebuild with the step-1 public-key-only `--image-options` TOML plus `--vendor-sig-override vsig.bin --owner-sig-override osig.bin`. Alternatively, patch the bytes at offsets 4,444 and 11,856 of the step-1 bundle.
   - Rebuilding is only valid if the build is reproducible. caliptra-sw has a consistency test for ELF builds, but this was not checked here.
5. Verify before fusing:
   - Check the ECC signatures with your public keys over the §3 byte ranges: ECDSA P-384 over SHA-384 of bundle[16588:16708] (vendor) and bundle[16588:16748] (owner). This is the check that passed on the fake-key bundle.
   - Boot in Verilator (SIMULATION_2X.md), or in caliptra-sw's emulator, with the regenerated fuse image.

No other external-signing tool exists in caliptra-sw `dd1e79d`: no PKCS#11 / HSM plug-in.

6.4 Regenerate the fuse image

- `python3 caliptra2x/scripts/gen_fuse_image_2x.py path/to/your-bundle.bin`. This recomputes `FUSE_VENDOR_PK_HASH` and `CPTRA_OWNER_PK_HASH` from your keys and sets `FUSE_PQC_KEY_TYPE`.
- Cross-check against `caliptra-image-app … --print-hashes` (§6.3 A) if you build with method A.
- In a product:
  - `FUSE_VENDOR_PK_HASH`, `FUSE_PQC_KEY_TYPE`, revocations and SVNs are burned into OTP at manufacturing.
  - `CPTRA_OWNER_PK_HASH` (a register in 2.x) is written and locked from immutable SoC logic or ROM on every cold boot, from owner-controlled storage.
  - The UDS seed and field entropy are generated per device and protected by the obfuscation key. They are **never** the STUDY formulas in the script.
- The script currently asserts bundle SVN = 0. Change its SVN handling (and the fuse values) when you adopt an SVN policy (§6.5).

6.5 SVN and revocation policy

- **FW SVN:**
  - Header `svn` is set by `fw_svn` (`--fw-svn` or the image-options `fw_svn`). Maximum 128 (`MAX_FIRMWARE_SVN`).
  - The fuse SVN is `128 − leading_zeros(FUSE_RUNTIME_SVN[0..3] as u128)`, i.e. the position of the highest burned bit (`drivers/src/fuse_bank.rs`, `first_set_msbit`). Burning bit $n-1$ sets fuse SVN n.

  - The ROM rejects images with `svn` < fuse SVN, unless `FUSE_ANTI_ROLLBACK_DISABLE` is set (keep it 0 in production).
  - Bump the SVN only for security fixes, and burn the next bit only after the new image is deployed fleet-wide.
- **Vendor key revocation:**
  - `FUSE_ECC_REVOCATION` (4-bit mask), `FUSE_MLDSA_REVOCATION` or `FUSE_LMS_REVOCATION` (masks); bit *i* revokes vendor key *i*.
  - The last index cannot be revoked.
  - After revoking key *i*, sign with key *i*+1 (`ecc_key_idx` / `pqc_key_idx`).
- **Owner key rotation:** sign with the new owner key, and update the owner hash the SoC writes into `CPTRA_OWNER_PK_HASH`. It is not a fuse in 2.x, so this needs no OTP burn, but the storage must be authenticated.
- `FUSE_FMC_KEY_MANIFEST_SVN`, `FUSE_SOC_MANIFEST_SVN` and `FUSE_SOC_MANIFEST_MAX_SVN` are written 0 in this study.

6.6 Keep keys out of git

- The repo `.gitignore` now ignores `keys/`, `*.key`, `*_priv*.pem`, `*_priv*.bin`, `*.p8`, `*.p12` and `*.pfx`. Even so, never copy private keys into the tree. Keep them in the HSM and keep only public keys and bundles in git.
- The fake-key private halves are already public upstream; they are **not** copied into this repo.
- Review `git status` before every commit when working with real keys.

---

7. Checklist before you trust a bundle

1. `sha256sum -c caliptra2x/fw_2x/SHA256SUMS` passes (or record new sums).
2. The bundle parses with marker `CMN2`, the expected `pqc_key_type`, SVN and TOC sizes (the §2 table, or the asserts in `gen_fuse_image_2x.py`).
3. The fuse image was regenerated from **this** bundle, and both printed hashes match your HSM's public keys.
4. Verilator boot reaches `[fwproc] Img verified …` and `ready_for_runtime` with that fuse image.
5. No private key is in the working tree (`git status`, `git ls-files | grep -i priv`).